

Article

Schema Retrieval with Embeddings and Vector Stores Using Retrieval-Augmented Generation and LLM-Based SQL Query Generation

Mehmet Bozdemir and Metin Bilgin *

Department of Computer Engineering, Bursa Uludağ University, 16059 Bursa, Turkey;
512231003@ogr.uludag.edu.tr

*Correspondence: metinbilgin@uludag.edu.tr

Abstract

In today's world, where the volume and variety of data are increasing at an extraordinary rate, extracting meaningful insights from data is of critical importance; however, the complexity of standard database query languages makes it difficult for users without technical expertise to access information. This study proposes an innovative Retrieval-Augmented Generation (RAG) architecture that analyzes natural language queries, identifies related database schemas, and automatically converts them to SQL. Unlike fixed schema selection (fixed-k) methods, a unique hierarchical clustering mechanism is introduced to dynamically determine the number of relevant schemas, minimizing noise. Furthermore, the architecture incorporates an iterative repair mechanism, data enrichment with sample rows, and a hybrid query strategy (Turkish + English) to overcome cross-lingual barriers. Performance evaluations on 15 databases demonstrate that the proposed method improved the schema retrieval F1 score from 0.79 to 0.88. In the SQL generation phase, the execution accuracy (EX) of the GPT-4o model increased from 0.70 to 0.78 with the proposed optimizations, representing an approximate 11% improvement relative to the baseline configuration without requiring fine-tuning.

Keywords: embedding; LangChain; large language models; RAG; schema retrieval; Text-to-SQL; vector database

1. Introduction

Language modelling began in the 1950s with Shannon's application of information theory to language and has since formed the basis of fields such as natural language processing, speech recognition, and machine translation [1]. Large Language Models (LLMs) are artificial intelligence systems designed to generate human-like text. The emergence of these systems is rooted in a 2017 paper that marked a major turning point in natural language processing [2]. The transformer architecture described in the paper laid the foundation for today's large language models. The first widely recognised LLM was the GPT-2 model developed by OpenAI (San Francisco, CA, USA) in 2019. This model was much larger than all previously developed models, with 1.5 billion parameters. Furthermore, this model was used in many Natural Language Processing (NLP) tasks without undergoing fine-tuning [3]. In subsequent years, OpenAI released GPT-3, GPT-3.5 (Chat GPT), GPT-4o and, most recently, the GPT o1-preview model [4]. Companies such as Google

Academic Editors: Yu Sun and
Chengliang Chai

Received: 11 December 2025

Revised: 1 January 2026

Accepted: 5 January 2026

Published: 6 January 2026

Copyright: © 2026 by the authors.
Licensee MDPI, Basel, Switzerland.
This article is an open access article
distributed under the terms and
conditions of the [Creative Commons
Attribution \(CC BY\) license](https://creativecommons.org/licenses/by/4.0/).

(Mountain View, CA, USA), Meta Platforms, Inc. (Menlo Park, CA, USA) and Microsoft Corporation (Redmond, WA, USA) have joined this field pioneered by OpenAI and created their own language models.

Retrieval-Augmented Generation (RAG) is an approach that enhances the accuracy and reliability of large language models by incorporating data from external sources. It is frequently used in situations where LLM access to information is limited [4]. RAG has gained popularity among large language models in recent years. Natural language processing studies conducted with language models requiring parametric memory present certain challenges. Model training presents users with a process that is quite intensive in terms of time and cost. Since training parametric models is lengthy and time-consuming, memory-free approaches such as RAG have come to the fore [5]. With RAG, the necessary information is drawn from external sources, effectively solving problems such as time and cost that arise during model training. With RAG, the information provided to the large language model is enriched, ensuring more accurate and consistent results. This has also largely prevented one of the biggest problems of large language models, namely hallucination [6].

In the light of these advancements, the motivation behind this work is to establish the necessary infrastructure to enable individuals who are not experts in the SQL language to perform various manipulation operations in database management systems. To this end, we aim to automatically generate the corresponding SQL statement when end users enter a query expressed in natural language into the system. The generated SQL query is executed simultaneously on the relevant database, and the results can be presented to the user in the form of SQL code or a visual interface (View). The data associated with the query expressed in natural language is obtained through a vector database containing database schemas [7].

However, it is important to be aware of the limitations of Large Language Models (LLMs). LLM-based approaches can sometimes generate incorrect or contextually inappropriate queries. Therefore, rather than presenting the system to the end user as a completely secure and error-free automation, it would be more appropriate to position it as a learning tool or an efficiency-enhancing support system. The aim of this study is not for LLM to replace expertise, but rather to facilitate access to areas requiring expertise and simplify user interaction with the system.

In this study, three fundamental challenges affecting success in the Text-to-SQL process were addressed. The first challenge is determining how many schemas a natural language query is associated with, as a query may be related to a single table or encompass multiple tables. We utilized a hierarchical clustering algorithm to determine the number of schemas. With this algorithm, schemas from the vector database were grouped according to similarity scores, and the number of clusters was dynamically determined by selecting the group with the highest scores. Thus, noise was minimised by providing the Large Language Model (LLM) with only the relevant schemas from the database, rather than all of them. The more concise and relevant data provided to large language models, the greater the accuracy of the results obtained.

The second challenge arises from the specific cross-lingual scope of this research. Unlike standard monolingual benchmarks, this study addresses a more complex scenario where natural language queries are expressed in Turkish, while the underlying database schemas and metadata are defined in English. To overcome this language mismatch, multilingual embedding models were employed alongside a hybrid query strategy, where Turkish queries and their English translations were vector-concatenated. This ensured that semantically consistent schemas were retrieved from the vector database despite the language barrier.

Thirdly, schema names and column information alone are sometimes insufficient for resolving complex queries and understanding the content of the data. Even though the model knows the table structure, not knowing the format or type of data within the cells can lead to incorrect SQL generation. To address this issue, schema definitions were enriched with sample data rows to provide the model with explicit data content context.

In this context, the study presents an innovative approach that incorporates clustering methods and data enrichment strategies to generate SQL queries based on natural language expressions. The original contributions of the study to the literature can be summarized as follows:

1. **Dynamic Schema Selection:** A hierarchical clustering algorithm has been innovatively applied to determine how many schemas a natural language query is associated with. This method groups schemas from the vector database according to similarity scores, enabling only the most relevant schemas to be provided to the model.
2. **Overcoming the Language Barrier:** When queries are in Turkish and database schemas are in English, semantic matching success has been increased by using multi-language embedding models.
3. **Hybrid Enrichment Strategy:** To improve the system's performance, a multi-layered approach has been developed, where sample data rows extracted from the database are added to the schema definitions (data enrichment) and supported by English translations of Turkish natural language queries (hybrid query). This ensures that the model understands not only the schema structure but also the data content, significantly improving the overall success of the methodology.

To address the limitations identified in the literature, this study aims to develop a novel RAG architecture enhanced with hierarchical clustering and hybrid querying strategies specifically for Cross-Lingual Text-to-SQL tasks. Consequently, this research addresses the following research questions (RQs):

RQ1: Can hierarchical clustering effectively reduce schema noise and dynamically identify relevant schemas in RAG-based Text-to-SQL tasks?

RQ2: Does a hybrid (Turkish–English) query strategy outperform monolingual retrieval in cross-lingual scenarios?

RQ3: Can the execution accuracy of generalist LLMs be improved without fine-tuning through iterative self-correction and data enrichment?

The remainder of this paper is organized as follows: Section 2 reviews related works and the current state of Text-to-SQL research. Section 3 details the materials and proposed methods, including the clustering algorithm and RAG architecture. Section 4 presents the experimental results and performance comparisons. Finally, Section 5 offers conclusions and discusses future research directions.

2. Related Works

This section presents information on scientific studies conducted in the relevant field.

A comprehensive review of the literature indicates a significant gap regarding the application of RAG-based Text-to-SQL generation for the Turkish language. This constitutes one of our most important motivations. The studies conducted are generally based on the principle of training or fine-tuning a language model. However, recent comprehensive surveys highlight a significant paradigm shift in this domain. These studies indicate that the field is rapidly moving from traditional rule-based and fine-tuning approaches towards exploiting the zero-shot, in-context learning, and retrieval-augmented generation (RAG) capabilities of Large Language Models (LLMs) [8–10]. From this perspective, to the best of our knowledge, this is the first study to investigate Text-to-SQL schema retrieval for Turkish databases using a RAG-based approach.

Kanburoğlu and Tek examined the conversion of natural language to SQL in their study. The dataset they used is called Text-to-SQL and is in English. Due to the lack of a Turkish dataset, they prepared a dataset called TUR2SQL consisting of 10,809 natural language and SQL equivalents. Using the SQLNet and ChatGPT models, they conducted various studies on the TUR2SQL dataset. They demonstrated that ChatGPT performed better on the dataset used [11].

Another study conducted by Kanburoğlu and Boray examined the conversion of natural language expressions into SQL. They used the WikiSQL and Spider datasets in their work. They measured the system's success by training models such as SQLNet, RAT-SQL, and Seq2SQL. On the WikiSQL dataset, the SeaD model achieved 92.9% execution accuracy. On the Spider dataset, the T5-SR-3b model achieved 79.9% exact match accuracy [12].

Wang et al. developed a system that automatically generates SQL queries from natural language expressions. The study focused particularly on reducing errors that arise when models generate SQL queries and introduced a new mechanism called "Execution-Guided Decoding" (EG). This mechanism detects erroneous SQL queries and ensures that more accurate and valid SQL queries are generated. They tested the EG mechanism on four different models. These models were evaluated on datasets such as WikiSQL, ATIS, and GeoQuery. They reported that EG improved performance across all these models and increased execution accuracy by between 1% and 6% [13].

Kumar et al. introduced BookSQL, a large-scale Text-to-SQL dataset, citing a lack of data, particularly in the field of finance. While general-purpose datasets (e.g., Spider, WikiSQL, BIRD-SQL) cover a wide range of domains, datasets specific to particular fields, such as accounting, are limited. To fill this gap, they introduced their own accounting dataset consisting of 100,000 query-SQL pairs. They tested language models such as RESDSQL, UniSAr, and GPT-4 using this dataset. The results revealed significant performance losses in queries specific to the accounting field. For example, while the RESDSQL model achieved 80.5% Exact Match Accuracy (EMA) on the Spider dataset, it only achieved 10.8% EMA on the BookSQL dataset [14].

In their study, Rushdy and Thayasivam proposed a noise filtering mechanism for a T5-based model to improve the performance of models used in the task of converting to SQL queries. The study presents an approach that accepts the most suitable schema as input and enables the generation of target SQL queries. A classifier structure based on the RoBERTa model was used, and this classifier matches schema elements with natural language queries and passes them to the T5 model. The proposed method was tested on the Spider dataset and achieved an Exact Match (EM) Accuracy of 72.7% and an EX success rate of 80.2% [15].

In the study developed by Demirkiran and colleagues, the T5, SQLCoder, and GPT-3.5 Turbo models were fine-tuned using the TUR2SQL dataset. The dataset consists of a structure combining queries and schemas in Turkish natural language. When the data was fine-tuned with the specified models, an EX success rate of 97% was achieved with SQLCoder (With Schema Context), 98% with T5 (With Schema Context), and 94% with GPT-3.5 Turbo [16].

The study conducted by Usta et al. was carried out using DBTagger, which involves combining the query with the relevant tables and columns in the database. This mapping process ensures the accurate analysis of the word-based context and the database schema. The input Natural Language Query (NLQ) is processed word-by-word, and tags related to the database schema are assigned to each word. This tagging enables the machine to learn which word corresponds to which schema component. This process was implemented using a Bi-Directional GRU-based model. In short, it ensures the association of the input natural query with the database schema and columns. This approach involves a model training process [17].

Among the studies conducted in the field of Text-to-SQL, methods such as SQLNet, RAT-SQL, DBTagger, and ReFSQL stand out. For example, ReFSQL offers a special Retrieval-Augmented Framework to reduce structural errors, while SQLNet is a powerful model for generating SQL queries directly from natural language expressions. These studies have generally been tested on English datasets (e.g., Spider, WikiSQL) and have achieved high accuracy rates. The lack of Turkish language datasets is a significant problem encountered in this field. Our study aims to develop a method that converts Turkish natural language queries into SQL queries to address this deficiency.

The work of Demirkiran and colleagues is similar to ours in that it was performed on Turkish data. However, in our work, the RAG method is used, and the relevant schemas are obtained from the vector database via vector similarities (cosine similarity). These retrieved schemas are processed in the query context using a large language model. Our work does not involve any model training, which distinguishes it from similar studies in the existing literature.

The DBTagger model developed by Usta and colleagues aims to identify which schemas a query is associated with. In this respect, it serves a similar purpose to our work. However, the DBTagger model uses a deep learning method called Bi-Directional GRU to determine the relevant schemas and therefore involves a model training process. In contrast, our work does not involve any model training; schemas are obtained based on similarity scores calculated in the vector database using the RAG method. This approach differentiates our work from similar studies in the literature.

As can be seen from the studies conducted, natural language processing models are frequently used to generate SQL queries. In particular, models such as T5 and BART, which are designed for seq2seq tasks, or language models developed specifically for SQL, hold an important place in this field. For example, Zhang et al. [18] developed a T5-based model to achieve high accuracy rates in Text-to-SQL conversion. In this study, the T5 model was fine-tuned on datasets such as WikiSQL and Spider. Furthermore, a structure was developed with the BERT model to learn correct and incorrect schemas, and this structure ensured the selection of correct schemas. However, the successful performance of these models relies on large amounts of data and computational power; moreover, they generally require a special training process.

Such approaches require training language models, and this process is both time-consuming and resource-intensive. However, this study does not involve such a model training phase. Instead, SQL queries are generated directly using the RAG system. The RAG system is a powerful approach that converts natural language inputs into target outputs using information retrieved from the knowledge base, eliminating the need for model training. This perspective is supported by recent surveys [9,10], which classify Text-to-SQL approaches into fine-tuning and prompt engineering, noting that prompt-based methods offer superior flexibility and lower resource consumption for domain adaptation. Similar schemas are grouped using a clustering method, the cluster with the highest score is selected, and this cluster is sent to the LLM together with the query to generate the SQL query. This method offers a lighter and more flexible solution without model training.

The fundamental difference between this work and previous literature is that SQL queries are generated using only the RAG system, without requiring any seq2seq language model training. This method minimises resource usage and offers a more specialised and practical solution for specific databases by focusing on the selection of correct schemas. Effectively demonstrating the potential of the RAG system in SQL query generation, this work makes a significant contribution to the literature by avoiding the difficulties associated with model training and offering a more flexible and resource-efficient solution using clustering methods for correct schema selection.

In addition to these approaches, recent advancements in 2024 and 2025 have shifted towards sophisticated decomposition and framework-based strategies to address the complexity of large-scale databases. For instance, MAC-SQL [19] introduced a multi-agent collaborative framework that utilizes specialized agents for decomposition and refinement, establishing a new state-of-the-art on the BIRD benchmark. Similarly, DTS-SQL [20] proposed a two-stage decomposed fine-tuning method, demonstrating that small open-source models (e.g., 7B parameters) can rival proprietary LLMs by explicitly separating schema linking from SQL generation. Furthermore, the practical utility of RAG-based frameworks like Vanna.ai and LlamaIndex was evaluated in a recent industrial case study [21]. This study highlighted that while AI tools significantly improve productivity compared to human analysts, their performance drops notably on real-world, unannotated datasets compared to synthetic benchmarks (e.g., Spider). Although these studies represent the cutting edge in Text-to-SQL, they predominantly focus on English-centric tasks. Applying such advanced prompting and framework strategies directly to agglutinative languages like Turkish, without addressing the specific cross-lingual semantic gap, remains an open research challenge.

Based on the gaps identified in the literature, particularly the lack of training-free and cross-lingual solutions for Turkish, we propose the following hypotheses:

H1: *Dynamic schema selection via hierarchical clustering will effectively minimise noise and adapt to varying schema counts, thereby yielding high F1 scores in schema retrieval tasks (Addressing RQ1).*

H2: *A hybrid (Turkish–English) query strategy will improve semantic matching capabilities in cross-lingual retrieval tasks compared to monolingual approaches (Addressing RQ2).*

H3: *Enriching prompts with sample data and applying iterative self-correction will significantly increase the Execution Accuracy (EX) of generalist LLMs without fine-tuning (Addressing RQ3).*

3. Materials and Methods

This section details the processes of preparing the dataset, determining suitable embedding models for the RAG architecture, and establishing the vector database structure. The proposed hierarchical clustering algorithm for identifying the most relevant candidate schemas retrieved from the vector database and the data enrichment and hybrid query strategies applied to enhance the system's performance are explained. Finally, the SQL generation phase using Large Language Models (LLMs) and the evaluation metrics and testing methods used to measure the accuracy of the results obtained are presented.

3.1. Preparation of the Dataset

To test the system developed in this study, databases contained within the open-source BIRD [22] and Spider [23] data sets were used. Since these datasets serve as the standard benchmarks for cross-domain Text-to-SQL tasks in the literature and are fundamentally provided in SQLite format, all performance evaluations in this study were conducted within the SQLite environment. Fifteen of the 245 SQLite databases contained within these data sets were utilised. Table 1 shows information about these databases. A total of 1006 queries from the 15 databases were used. The datasets containing these queries were obtained from the HuggingFace ecosystem [24]. The natural language queries in the datasets are in English. The natural language queries were initially translated into Turkish using a neural machine translation engine. Subsequently, a comprehensive manual review and post-editing process were conducted to ensure semantic fidelity, resolve

ambiguities, and rectify any expressions that could compromise the accuracy of SQL generation. Furthermore, the “Related_schemas” column, which was used as reference data (ground truth) to measure schema retrieval success and was not present in the original dataset, was created using a special Python (version 3.12.3) script that analyses SQL queries. In this process, each SQL query was parsed to identify related table names, and then column definitions and constraints (metadata) for these tables were retrieved from the SQLite database to create schema information. The logic and processing steps for creating the “Related_schemas” column are summarised in Algorithm 1.

Algorithm 1 Extracting Related Schemas from SQL Queries

Require: SQL_Query, SQLite_Database

Ensure: Related_Schemas

- 1: Parse SQL_Query to extract table names using keywords: **FROM, JOIN, UPDATE, INSERT INTO, DELETE FROM**
 - 2: **For all** Table_Name in extracted list **do**
 - 3: Connect to the SQLite database
 - 4: Execute **PRAGMA table_info(Table_Name)** to retrieve column definitions.
 - 5: Execute **PRAGMA foreign_key_list(Table_Name)** to retrieve foreign key constraints.
 - 6: Construct a **CREATE TABLE** statement based on the retrieved metadata.
 - 7: **end for**
 - 8: Concatenate all **CREATE TABLE** statements.
 - 9: Return the result as the Related_Schemas text.
-

Table 1. Databases.

Database Name	Number of Queries	Description
architecture	17	This database contains information about architects and their designs. It has three schemas: “architect,” “bridge,” and “mill.”
cars	82	This database contains information about cars. It has four schemas: “country,” “price,” “data,” and “production.”
chinook	84	This database contains data related to the music industry. It has 11 schemas: “Album,” “Artist,” “Customer,” “Employee,” “Genre,” “Invoice,” “InvoiceLine,” “MediaType,” “Playlist,” “PlaylistTrack,” and “Track.”
coffee_shop	18	This database contains customer and member information for a coffee shop. It has four schemas: “shop,” “member,” “happy_hour,” and “happy_hour_member.”
company_employee	16	This database contains information about companies and their employees. It has three schemas: “people,” “company,” and “employment.”
college	170	This database contains information about courses, classes, departments, faculty members, and students belonging to a university. It has 11 schemas, including “classroom,” “department,” “course,” “instructor,” “section,” “teaches,” “student,” “takes,” “advisor,” “time_slot,” and “prereq.”
company_office	40	This database contains information about companies and office buildings. It has three schemas: “buildings,” “Companies,” and “Office_locations.”
customer_complaints	46	This database contains information about customer complaints and products. It has four schemas: “Staff,” “Customers,” “Products,” and “Complaints.”
department_store	88	This database contains information related to a chain of stores. It has 14 schemas, including “Addresses,” “Staff,” “Suppliers,” “Department_Store_Chain,” “Customers,” “Products,” “Supplier_Addresses,” “Customer_Addresses,” “Customer_Orders,” “Department_Stores,” “Departments,” “Order_Items,” “Product_Suppliers,” and “Staff_Department_Assignments.”

election	68	This database contains information related to elections. It has three schemas: "county," "party," and "election."
movie	98	This database contains information about movies and reviews. It has three schemas: "Movie," "Reviewer," and "Rating."
sakila	82	This database contains information related to movie rental transactions. It has 14 schemas, including "actor," "address," "category," "city," "country," "customer," "film," "film_actor," "inventory," "language," "payment," "rental," "staff," and "store."
soccer	106	This database contains information about football players, teams, and leagues. It has seven schemas: "Player_Attributes," "Player," "League," "Country," "Team," "Team_Attributes," and "sqlite_sequence."
social_media	76	This database contains information related to social media (especially Twitter). It has three schemas: "location," "user," and "Twitter."
voter	15	This database contains information related to voting processes and contestants. It has three schemas: "AREA_CODE_STATE," "CONTEST-ANTS," and "VOTES."
Total	1006	

The "Related_schemas" column shown in Table 2 was used as reference (ground truth) data to measure the model's retrieval success. In this context, similar schemas were retrieved from the SQLite database via the RAG system using the Turkish queries in the "Question_tr" column, and the accuracy of these outputs was compared with the information in the "Related_schemas" column. Similarly, the performance of the SQL outputs generated by the LLM was tested using the reference queries in the "SQL" column. All databases in the dataset were tested using Navicat database management software. During this process, some data was found to be missing, and some queries were found to be incorrect. Missing data was supplemented with synthetic (artificial) data, and incorrect SQL queries were corrected to optimise the dataset.

Table 2. Example of the Dataset Used.

Question_en	Question_tr	SQL	Related_Schemas
How many architects are female?	Kaç mimar kadın'dir?	SELECT count(*) FROM architect WHERE gender = 'female'	CREATE TABLE architect (id TEXT PRIMARY KEY, name TEXT, nationality TEXT, gender TEXT);
What is the maximum length in meters for the bridges, and what are the architects' names?	Köprülerin maksimum uzunluğu metre cinsinden ne kadardır ve mimarların isimleri nelerdir?	SELECT max(T1.length_meters), T2.name FROM bridge AS T1 JOIN architect AS T2 ON T1.architect_id = T2.id	); CREATE TABLE bridge (architect_id INT, id INT PRIMARY KEY, name TEXT, location TEXT, length_meters REAL, length_feet REAL, FOREIGN KEY (architect_id) REFERENCES architect(id)

);

While preparing the dataset, various issues were encountered due to “language mismatch” caused by natural language queries being in Turkish and database content being in English. These issues and the applied solution methods are detailed below:

Value Translation Error: Examining the first row of Table 3, it can be seen that the word “database” in the original query has been translated into Turkish as “veritabanı”. However, when the SQL statement is checked, it is understood that the value of the “course_name” variable in the database is stored in English as “database”. When the Turkish query is given to the LLM, the model creates a condition such as “course_name = Veritabanı”, and since this value is not found in the database, the query returns empty results. To prevent such errors, queries have been revised as in Table 3, preserving proper names and values in the database. A similar correction has been applied in the example in the third row.

Table 3. Incorrect and Corrected Natural Language Queries.

No.	English Query	Incorrect Query (EN)	Incorrect Query (TR)	Corrected Query (EN)	Corrected Query (TR)	SQL
1	What are the descriptions of the courses named “Database”?	What are the descriptions of the courses named “veritabanı”?	“Veritabanı” isimli derslerin açıklamaları nelerdir?	What are the descriptions of the courses named “database”?	“database” isimli derslerin açıklamaları nelerdir?	SELECT course_description FROM COURSES WHERE course_name = “database”
2	Tell the origin country of car no.382.	Tell the country of car no.382.	382 numaralı arabanın menşesi ülkesini söyleyin.	Tell the unique origin country of car no.382.	382 numaralı arabanın benzersiz menşesi ülkesini söyleyin	SELECT DISTINCT T2.country FROM production AS T1 INNER JOIN country AS T2 ON T1.country = T2.origin WHERE T1.ID = 382
3	return me the abstract of “Making database systems usable”	Return the abstract of the topic “Making database systems usable”.	“Veritabanı sistemlerini kullanılabilir hale getirme” konusunun özetini bana geri gönderin.	List the abstract of the title “Making database systems usable”.	“Making database systems usable” adlı başlığı listele	SELECT abstract FROM publication WHERE title = “Making database systems usable”;
4	What is the average number of stars each reviewer awards for a movie?	What is the average number of stars each reviewer gave to a movie?	Her eleştirmenin bir filme verdiği ortalama yıldız sayısı nedir?	List both the average number of stars and the reviewer names for each movie.	Her eleştirmenin bir filme verdiği ortalama yıldız sayısını ve ismini listele	SELECT T2.name, avg(T1.stars) FROM Rating AS T1 JOIN Reviewer AS T2 ON T1.rID = T2.rID GROUP BY T2.name

Ambiguity and Missing Keywords: In some cases, ambiguity in the Turkish expressions generated after translation caused the LLM to use incorrect SQL operators. For example, the reference SQL query in the second row of Table 3 contains the term “DISTINCT”. However, the Turkish query does not clearly state that unique records are required, causing the LLM to miss this detail. To resolve this issue, the Turkish queries have been rewritten to clearly describe the desired result.

Missing Column Display: The English query in the fourth row of Table 3 has been translated into Turkish as “What is the average number of stars each critic gave to a film?”

“Based on this statement, the LLM generates an SQL query that only returns the Average Star Rating (AVG). However, upon examining the reference SQL query, it is evident that both the average rating and the film title should be listed. Therefore, the Turkish query has been corrected to include all expected columns.

As a result, 1006 queries within 15 databases were manually reviewed one by one, and the semantic inconsistencies mentioned above were resolved, resulting in the final version shown in Table 3.

3.2. Embedding Model Selection

To perform RAG, an embedding and a vector database are required. This section discusses how the embedding vector is selected. Word embedding essentially expresses that in natural language processing, words are not merely numbers; each word carries meaning based on proximity or distance relationships within the vector space. Capturing such meaningful relationships between words has had a revolutionary impact in the field of natural language processing [25]. This is because these relationships have enabled more meaningful and accurate outputs. There are many different word vectors used to represent words: Glove, Word2Vec, etc.

When considering a model such as Word2Vec, it is seen that word representation vectors are obtained using approaches such as skip-gram [26] and CBOW [27]. While the skip-gram algorithm attempts to predict the surrounding words by looking at the central word, the CBOW algorithm attempts to find the central word based on the surrounding words. Using such a system, models are created to find the numerical equivalent of a given word in the vector space.

The main challenge of the system developed in this study is that user queries are in Turkish, while the target database schemas and data are in English (Cross-Lingual Retrieval). Therefore, when selecting the embedding model, multilingual support and cross-lingual semantic alignment capability were determined to be the most critical criteria.

In the first phase of the study (Baseline), the following three basic models, which are widely used in the literature and have multilingual support, were tested:

OpenAIEmbeddings (text-embedding-3-large): A commercial model developed by OpenAI that captures high-dimensional and contextually rich information.

CohereEmbeddings (embed-multilingual-v3.0): A commercial model specifically optimised for multilingual tasks, particularly excelling in cross-lingual search performance.

HuggingFace (sentence-transformers/paraphrase-xlm-r-multilingual-v1): An open-source model based on the XLM-RoBERTa architecture, widely used by the community.

In the second phase of the study's optimisation, the latest and highest-performing models in the literature were incorporated into the system to further enhance retrieval success. In this context, the “voyage-3-large” model, which performs particularly well in Turkish–English pairings, and the “ytu-ce-cosmos-turkish-e5-large” and “Qwen3-Embedding-8B” models, which have been specially trained for Turkish in the open-source world, were also added to the experiments for a comprehensive comparison. These selected models have both increased the system's semantic accuracy and minimised information loss due to language barriers.

3.3. Vector Database Selection

In RAG architecture, storing data converted into vector space using the embedding model and providing fast access is of critical importance. After database schemas are converted into embedding vectors, these vectors are stored in a vector database. During the query phase, the embedding vector of the user query expressed in natural language is extracted, and the closest vectors are searched for in the vector database according to the cosine similarity metric. As a result of this process, the schemas (or text fragments) with

the highest semantic relevance to the query are retrieved. Figure 1 shows the usage scheme of the vector database in the RAG system.

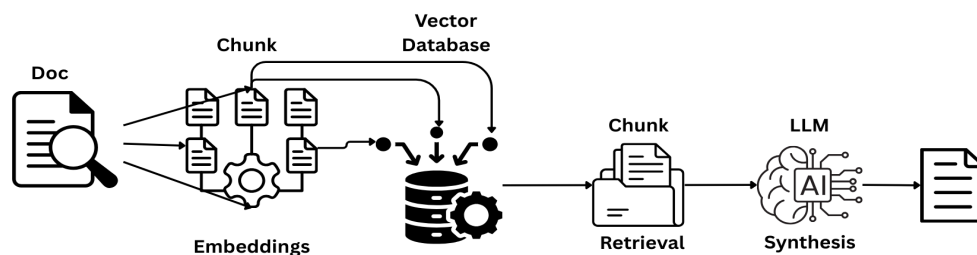


Figure 1. Document Parsing (Source: Created by the authors).

Various vector database solutions have been proposed in the literature to address different needs. In this study, the following three vector databases were used, considering their popularity, performance, and open-source community support:

- ChromaDB;
- Qdrant;
- Pinecone.

3.4. RAG (Retrieval Augmented Generation)

RAG is a popular method used to improve accuracy in situations where Large Language Models (LLMs) have limited access to information or require up-to-date/specialised domain knowledge. RAG aims to enable LLMs to produce more accurate and hallucination-free content by providing them with contextual information not found in the training data. RAG utilises information obtained to improve text generation. In Formula (1), X is the query to be searched for in the vector database. Considering the X context, the relevant K information is first retrieved from the corpus or database according to Formula 3. The retrieved information is then provided as input to the relevant prompt.

$$X = [x_1, x_2, \dots, x_n] \quad (1)$$

$$Y = [y_1, y_2, \dots, y_n] \quad (2)$$

$$K = \text{Retrieval}(X) \quad (3)$$

$$P(Y | X) = \prod_t P(y_t | X, K, y_1, \dots, y_{t-1}) \quad (4)$$

Y is the text generated as expressed in Formula (2). Prompt K directs the encoder to incorporate external information. According to Formula 4, the output value Y is calculated each time using the X and K data. The general formula is reached by multiplying the probabilities of all these values consecutively [28].

In this study, the RAG method has been designed on a two-stage structure for the purpose of converting natural language queries into SQL queries (Text-to-SQL). In the first stage, the most relevant database schemas corresponding to the natural language query are identified and retrieved from the vector database (Schema Linking). In the second stage, these retrieved schemas and the user query are combined and presented to the LLM, enabling the model to generate a valid SQL query using this structural context. This approach enables LLMs to dynamically recognise the database schema and generate SQL with high accuracy without the need for fine-tuning.

As illustrated in the workflow diagram in Figure 2, the process begins with fetching all schemas from the database and converting them into vector representations to populate the Vector Store. When a user submits a query, it is similarly vectorized, and similarity scores are calculated against the stored schemas. Crucially, instead of simply selecting the top-k results, an agglomerative clustering algorithm (highlighted in the red dashed box) groups the schemas based on their similarity scores. The system then automatically selects the cluster with the highest relevance scores to filter out noise. Finally, this refined schema context is fed into the Large Language Model (LLM) to generate the SQL query, which is then executed on the target database (e.g., MySQL, SQLite)

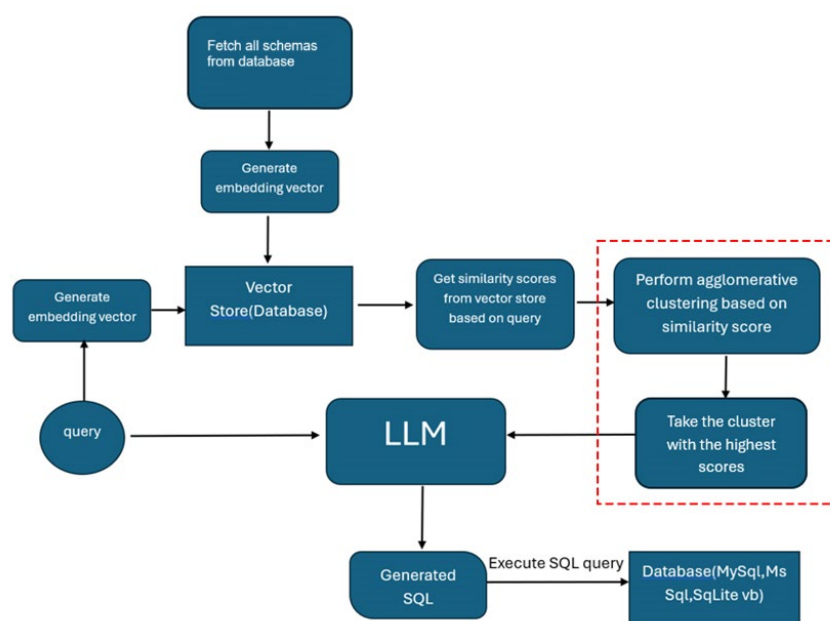


Figure 2. Clustering-Based RAG Workflow Diagram (Source: Created by the authors). (The red dashed box indicates the agglomerative clustering process.)

3.5. Determining the Number of Schemas Using a Hierarchical Clustering Algorithm

In Text-to-SQL problems, determining how many tables (schemas) a natural language query relates to is a challenging task. Standard RAG systems typically use a fixed k number (top- k retrieval). However, fixing the value of k can lead to errors: if $k = 3$ is selected, and the query only concerns 2 tables, then 1 irrelevant table (noise) is fed into the model; if $k = 1$ is selected, then a necessary table may be missing. This situation directly affects the success of the LLM. To overcome this problem, the Hierarchical Clustering Algorithm was used in the study [29].

This method determines a dynamic threshold value by analysing the distribution of similarity scores returned from the vector database. The schemas are initially ranked according to their Cosine Similarity scores (Equation (5)). To rigorously identify the separation between relevant and irrelevant schemas, these similarity scores (S) were treated as one-dimensional feature vectors and transformed into Cosine Distances ($D = 1 - S$). Consequently, the Agglomerative Clustering algorithm was implemented using the Ward linkage method and the Euclidean distance metric. In this one-dimensional context, the Euclidean metric effectively measures the ‘gap’ between the scores of consecutive schemas. The Ward method was selected because it minimizes the variance within clusters, ensuring the formation of compact and cohesive groups. The optimal number of clusters was dynamically determined by setting a distance threshold (cut-off) of 0.08 ($n_clusters = \text{None}$), which allows the model to automatically group relevant schemas without imposing a fixed cluster count. This specific threshold value of 0.08 was determined based on

empirical observations from preliminary experiments, as it yielded the highest stability in distinguishing relevant schemas from noise across the BIRD and Spider datasets. As an example, a query performed on the “Sakila” database and the similarity scores obtained are presented in Table 4.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} \quad (5)$$

Table 4. Cosine scores for the query (Sakila database).

Turkish Query	Cosine Scores
Hangi ilçelerin en az iki adresi var?	address (0.32)
	store (0.24)
	country (0.23)
	customer (0.22)
	city (0.22)
	staff (0.21)
	inventory (0.20)
	rental (0.20)
	film_text (0.19)
	actor (0.19)
	payment (0.18)
	category (0.18)
	film (0.18)
	language (0.17)
	film_actor (0.16)
	film_category (0.16)

The schemas grouped by the clustering algorithm are separated into distinct clusters based on their similarity scores. The aim here is to obtain the schemas in Cluster 0, which has the highest similarity score, as it represents the cluster formed by the schemas most similar to the query. The performance analysis of this clustering approach and the experimental results are detailed in Section 4.1.

3.6. Performance Improvement Strategies in the RAG System

At the baseline stage of the study, when examining the results obtained using standard embedding models, it was observed that schema access performance (F1 score: 0.79) reached saturation in complex and cross-lingual queries. To overcome this limitation and strengthen both the structural and semantic understanding of the model, a three-stage optimisation strategy was applied to the system.

Transition to Advanced Embedding Models: To enhance the system’s cross-lingual alignment capability, the most up-to-date models in the literature were tested. In this context, the following models were integrated into the system:

- “voyage-3-large”: A commercial model that performs well in multilingual retrieval tasks.
- “ytu-ce-cosmos-turkish-e5-large”: An open-source model optimised for Turkish-specific morphological and semantic structures.
- “Qwen3-Embedding-8B”: A model capable of in-depth context analysis with a large number of parameters.

Data Enrichment: In standard RAG approaches, embedding models are typically provided only with table and column names (metadata). However, the ambiguity of column names (e.g., uncertainty whether the “Rating” column refers to a film rating or age

restriction) sometimes makes it difficult for the model to perform accurate matching. In this study, schema definitions have been enriched with sample rows extracted from the database.

As shown in Table 5, in the “Enriched RAG” structure, four randomly selected rows of data from that table have been added below the column names. For example, for the Department column in the Students table, instead of only structural information, data such as “Computer Engineering” and “Architecture” were also presented to the model, enabling the LLM to understand the data content and format.

Table 5. Comparison of Standard RAG and Enriched RAG Structures.

Method	Input Data Structure (Prompt/Context)
Standard RAG	Table: Students (Student_ID, Name, Department, GPA) (Contains only structural information; no data content.)
Enriched RAG	Table: Students (Student_ID, Name, Department, GPA) Sample Rows: 1 (101, “Ahmet Yılmaz”, “Bilgisayar Müh.”, 3.5) 2 (102, “Ayşe Demir”, “Endüstri Müh.”, 3.8) 3 (103, “Mehmet Can”, “Mimarlık”, 2.9) 4 (104, “Fatma Kaya”, “Hukuk”, 3.1) (The model now understands from the data that “Bilgisayar Müh.” is a department.)

Advanced Hybrid Query Strategy: To enhance the semantic match between the query and the schema, a Hybrid Query (Turkish + English) strategy has been developed instead of monolingual (Turkish only) querying. In this process, the Turkish natural language query is translated into English. Then, instead of generating separate vectors, the texts are combined at the input level via text concatenation. Specifically, the Turkish query text (Q_{TR}) and the English translation (Q_{EN}) are joined with a separator token to form a unified input string ($Q_{Hybrid} = “Q_{TR} | Q_{EN}”$). This combined text is then processed by the embedding model to generate a single, semantically enriched vector representation (V_{Final}). This method preserves the semantic nuances of the source language while leveraging the terminology alignment of the target language within the model’s native vector space.

This method preserves the semantic features of both languages without information loss, and a semantic search is performed on the vector database using this enriched vector. This method aims to overcome the language barrier between English-based database schemas and Turkish queries. The applied strategy is exemplified below:

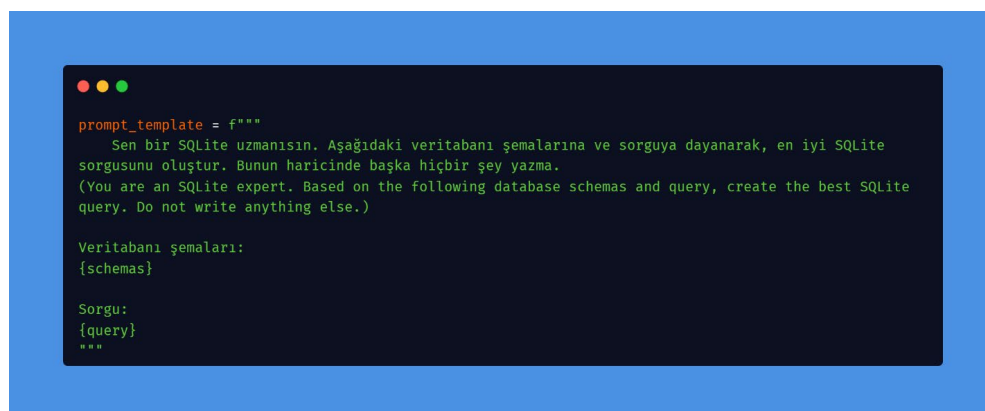
- Query-1 (Standard): TR: Köprülerin ortalama uzunluğu feet cinsinden nedir? (EN: What is the average length in feet of the bridges?)
- Query-2 (Hybrid): TR: Köprülerin ortalama uzunluğu feet cinsinden nedir? | EN: What is the average length in feet of the bridges?

3.7. Prompt Engineering and Iterative SQL Generation Strategy

This section details how schemas and user queries obtained from the RAG architecture are presented to Large Language Models (LLMs) using the LangChain ecosystem. Within the scope of the study, the SQLite query generation performance of the “GPT-4o”, “Gemini-1.5-Pro-001” and “Qwen1.5-110B” models was compared.

It is not always guaranteed that an LLM can directly generate a valid SQL query in its raw form; the model may sometimes add explanatory text or make syntactic errors. Therefore, the “System Role” assigned to the model and the constraints must be clearly

defined. The prompt structure given to the model at the initial stage is shown in Figure 3. In this prompt design, the model was assigned the role of “You are an SQLite expert” and the constraint “Only generate SQL code, do not provide explanations” was imposed.



```
prompt_template = f"""
Sen bir SQLite uzmanısın. Aşağıdaki veritabanı şemalarına ve sorguya dayanarak, en iyi SQLite
sorgusunu oluştur. Bunun haricinde başka hiçbir şey yazma.
(You are an SQLite expert. Based on the following database schemas and query, create the best SQLite
query. Do not write anything else.)

Veritabanı şemaları:
{schemas}

Sorgu:
{query}
"""
```

Figure 3. LLM Prompt 1.

However, it has been observed that even when correct schemas are provided, models sometimes generate erroneous SQL. To overcome this issue, the study developed an “Error-Based Iterative Repair” mechanism. The system follows these steps:

1. The first SQL query generated by the model is executed in the database.
2. If the query returns an error or an empty result, this error message and the faulty query are captured.
3. The model is retriggered with the “Retry Prompt” shown in Figure 4. This prompt provides the model not only with the schema and the query, but also with a list of “Previous faulty queries”.



```
prompt_template_last = f"""
Aşağıdaki veritabanı şemalarını(schemas), kullanıcı sorgusunu(query) ve geçmişteki hatalı sorguları
dikkate alarak yeni bir geçerli SQLite sorgusu oluştur. Geçmişteki hatalı sorguları tekrar etmeden,
sadece geçerli bir SQLite sorgusu döndür. Bunun haricinde başka hiçbir şey yazma.
(Based on the following database schemas, user query, and previous failed queries, create a new valid
SQLite query. Do not repeat any of the previous failed queries, and return only a valid SQLite query.
Do not write anything else.)

Geçmişteki hatalı sorgular: (Previous Failed Queries)
{previous_queries_str}

Veritabanı Şemaları:(Database Schemas)
{schemas}

Kullanıcı Sorgusu: User Query
{query}
"""
```

Figure 4. LLM Prompt 2.

This iteration process is repeated a maximum of three times. This limit was established as a heuristic threshold to prevent infinite error loops and reduce computational latency, as LLMs typically resolve syntactic errors within the first few attempts or tend to fail regardless of further iterations. Additionally, as a critical optimisation, the temperature parameter has been increased from 0 to 0.7 during the repair phase to enhance the model’s creativity. The quantitative impact of this temperature scaling strategy is analysed in Section 4.3.

3.8. Evaluation Metrics

To comprehensively evaluate the performance of the proposed RAG-based Text-to-SQL architecture, a multi-faceted assessment approach was employed. This section details the quantitative metrics used to measure the system's success in three critical dimensions: the effectiveness of the schema retrieval module in identifying relevant tables, the accuracy of the generated SQL queries in retrieving the correct data, and the semantic proximity of the predicted outputs to the reference standards. The definitions and calculation methodologies of these metrics are presented below.

Schema Retrieval Metrics (Precision, Recall, F1 Score): To measure the success of the Hierarchical Clustering algorithm in filtering relevant schemas from noise, standard information retrieval metrics were utilized.

- Precision: Measures the proportion of retrieved schemas that are genuinely relevant.
- Recall: Measures the ability to find all relevant schemas in the database.
- F1 Score: The harmonic mean of Precision and Recall.

These metrics are calculated using Equations (6)–(8):

$$Precision = \frac{TP}{TP + FP} \quad (6)$$

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (8)$$

where TP (True Positive) denotes correctly retrieved relevant tables, FP (False Positive) denotes irrelevant tables retrieved as noise, and FN (False Negative) denotes relevant tables that were missed.

Execution Accuracy (EX Score): This is the primary metric for Text-to-SQL tasks. It measures functional correctness by comparing the execution results of the generated SQL query with those of the ground truth SQL query. Unlike string matching, it validates whether the query retrieves the correct data. It is calculated as follows:

$$EX = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(V_{pred}^{(i)} = V_{gold}^{(i)}) \quad (9)$$

where N is the total number of queries, $(V_{pred}^{(i)}$ and $V_{gold}^{(i)})$ represent the execution results of the predicted and ground truth SQLs, respectively, and $\mathbb{I}(\cdot)$ is the indicator function which returns 1 if results match and 0 otherwise.

Table Similarity (Table Sim.) & SQL Similarity (SQL Sim.): Unlike traditional exact match metrics, these scores measure the semantic similarity between the generated output and the ground truth. They are calculated using the Cosine Similarity formula (defined in Equation (5)) between the embedding vectors of the predicted and reference texts. This approach evaluates how closely the meaning of the generated table names or SQL queries matches the reference, even if the wording differs slightly.

4. Results and Discussion

In this section, the experimental results of the proposed RAG-based Text-to-SQL system are presented in three main stages. First, the effectiveness of the Hierarchical Clustering algorithm for schema retrieval is analysed. Second, the baseline performance of three major LLMs (GPT-4o, Gemini-1.5-Pro, and Qwen-1.5-110B) is compared using standard RAG configurations. Finally, the impact of the proposed optimisation strategies (Data Enrichment and Hybrid Query) on model performance and schema complexity is evaluated.

4.1. Schema Retrieval Analysis Using Hierarchical Clustering

Before evaluating the full SQL generation pipeline, the effectiveness of the Hierarchical Clustering algorithm (described in Section 3.5) in identifying the correct number of schemas was analysed. Figure 5 illustrates the clusters formed based on cosine similarity scores for a sample query.

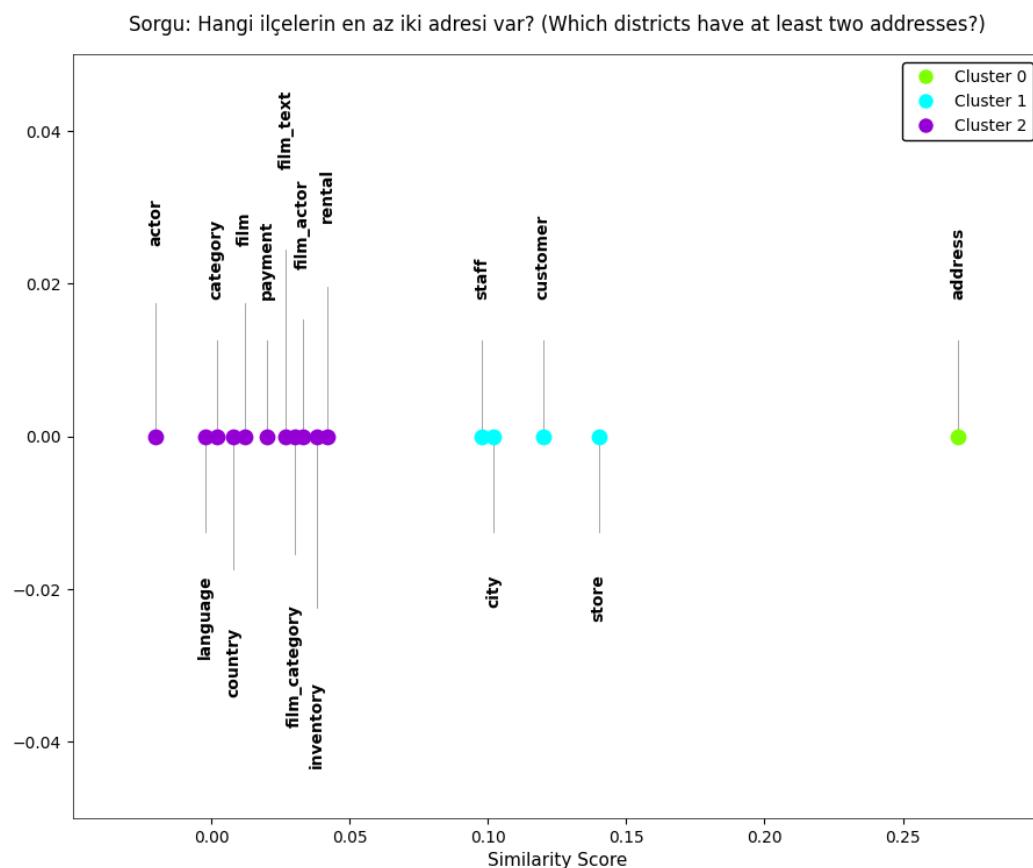


Figure 5. Clustering Result.

Using a clustering algorithm, three distinct clusters were formed based on the similarity scores shown in Figure 5. The vertical axis is used solely for visual organisation and does not represent any numerical value. The horizontal axis displays the schema similarity scores. The aim here is to obtain the schemas in Cluster 0, which has the highest similarity score. This is because Cluster 0 represents the cluster formed by the schemas most similar to the query in natural language. The other clusters have low scores and are therefore grouped in different areas. This developed system identifies the number of schemas most similar to a query. However, schemas similar to the natural language query cannot always be obtained with 100% accuracy. Nevertheless, it offers a powerful approach to identifying the number of schemas that were initially completely unknown. Since it is known that the query in Figure 5 is directly related to the “address” schema, 100% success was achieved in this case.

To demonstrate the effectiveness of this approach and the calculation of performance metrics, a specific query example was analysed:

Query:

Avustralya’da kaç şehir var? (How many cities are there in Australia?)

Clustering Results:

- Cluster 0: ['city (0.25)', 'address (0.23)', 'country (0.19)']

- Cluster 1: ['store (0.15)', 'film (0.14)', 'rental (0.14)', 'customer (0.13)', 'inventory (0.12)', 'staff (0.12)', 'actor (0.12)', 'category (0.11)', 'film_text (0.11)', 'payment (0.10)', 'language (0.10)', 'film_actor (0.10)', 'film_category (0.08)']

Truly Related Schemas:

- ['city', 'country']

In this specific case, the clustering algorithm successfully grouped the genuinely relevant “city” and “country” tables into Cluster 0. This successful retrieval of all required tables resulted in a perfect Recall score. However, the “address” table was also included in the cluster as an extra (False Positive), which negatively impacted the Precision score. To provide a balanced evaluation, the F1 Score (harmonic mean of Precision and Recall) was determined as the final success metric. The specific metric calculations for this sample query, based on the confusion matrix values (TP = 2, FP = 1, FN = 0), are presented in Table 6.

Table 6. Performance Metrics for the Sample Query (“How many cities are there in Australia?”).

Metric	Formula & Substitution	Value
Precision	$\frac{TP}{TP + FP} = \frac{2}{2 + 1}$	0.66
Recall	$\frac{TP}{TP + FN} = \frac{2}{2 + 0}$	1.00
F1 Score	$2 \times \frac{Precision \times Recall}{Precision + Recall} = 2 \times \frac{0.66 \times 1}{0.66 + 1}$	0.80

As seen in Table 6, while the system achieved 100% Recall, the intrusion of noise (the ‘address’ table) resulted in a Precision of 0.66, leading to a final F1 Score of 0.80.

Expanding this analysis to the general scope of the study, Precision, Recall, and F1 score values were calculated for a total of 1006 queries across 15 different databases. The tests covered nine different scenarios, including combinations of three different embedding models and three different vector databases. The comprehensive performance results for these scenarios are summarised in Table 7.

Table 7. Schema Matching Rates.

Embedding + Vector Database	Precision	Recall	F1
OpenAIEmbeddings(text-embedding-3-large) + ChromaDB	0.86	0.81	0.79
OpenAIEmbeddings(text-embedding-3-large) + Pinecone	0.74	0.90	0.76
OpenAIEmbeddings(text-embedding-3-large) + Qdrant	0.74	0.90	0.76
CohereEmbeddings(embed-multilingual-v3.0) + ChromaDB	0.85	0.82	0.78
CohereEmbeddings(embed-multilingual-v3.0) + Pinecone	0.72	0.91	0.74
CohereEmbeddings(embed-multilingual-v3.0) + Qdrant	0.71	0.91	0.74
HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1) + ChromaDB	0.79	0.56	0.63
HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1) + Pinecone	0.74	0.83	0.73
HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1) + Qdrant	0.74	0.83	0.73

When examining the embedding and vector databases used, the success rate of the “sentence-transformers” and “ChromaDB” pair in retrieving schemas was found to be lower than others, while the highest schema retrieval success was achieved with the pair formed by “text-embedding-3-large” and “ChromaDB”.

4.2. Schema Retrieval Performance (Hybrid Query Impact)

The impact of the Hybrid Query strategy and advanced embedding models on schema detection success is presented in Table 8.

Table 8. Schema Matching Rates (Improved Method).

Embedding + Vector Database	Precision	Recall	F1
VoyageAIEmbeddings (“voyage-3-large”) + Qdrant	0.92	0.89	0.88
VoyageAIEmbeddings (“voyage-3-large”) + Pinecone	0.92	0.89	0.87
VoyageAIEmbeddings (“voyage-3-large”) + ChromaDB	0.96	0.83	0.86
OpenAIEmbeddings(text-embedding-3-large) + Qdrant	0.89	0.89	0.85
OpenAIEmbeddings(text-embedding-3-large) + Pinecone	0.89	0.90	0.85
OpenAIEmbeddings(text-embedding-3-large) + ChromaDB	0.94	0.83	0.85
HuggingFaceEmbeddings (“ytu-ce-cosmos/turkish-e5-large”) + Qdrant	0.86	0.91	0.84
HuggingFaceEmbeddings (“ytu-ce-cosmos/turkish-e5-large”) + Pinecone	0.86	0.91	0.84
HuggingFaceEmbeddings (“ytu-ce-cosmos/turkish-e5-large”) + ChromaDB	0.92	0.84	0.84
HuggingFaceEmbeddings (“Qwen3-Embedding-8B”) + Qdrant	0.89	0.85	0.83
HuggingFaceEmbeddings (“Qwen3-Embedding-8B”) + Pinecone	0.89	0.85	0.83
HuggingFaceEmbeddings (“Qwen3-Embedding-8B”) + ChromaDB	0.92	0.80	0.82

Upon examining Table 8, it is evident that the proposed strategies significantly enhance performance. The combination of the “voyage-3-large” model and the Qdrant database achieved an F1 score of 0.88, demonstrating the highest performance. The “ytu-ce-cosmos-turkish-e5-large” model, on the other hand, achieved a competitive result with an F1 score of 0.84, coming very close to its commercial competitors (OpenAI, San Francisco, CA, USA; Voyage AI, Palo Alto, CA, USA).

4.3. Impact of Temperature Scaling and Iterative Repair

In addition to context enrichment, the study evaluated the model’s self-correction capability through the “Error-Based Iterative Repair” mechanism. A critical variable in this process is the temperature parameter, which controls the stochasticity of the LLM’s output.

As detailed in the methodology, the temperature was increased from 0 (deterministic) to 0.7 (creative) during the repair phase. This adjustment aims to prevent the model from repeating the same syntactic errors and encourages the exploration of alternative query structures. The quantitative impact of this strategy on the GPT-4o model across different databases is visualised in Figure 6.

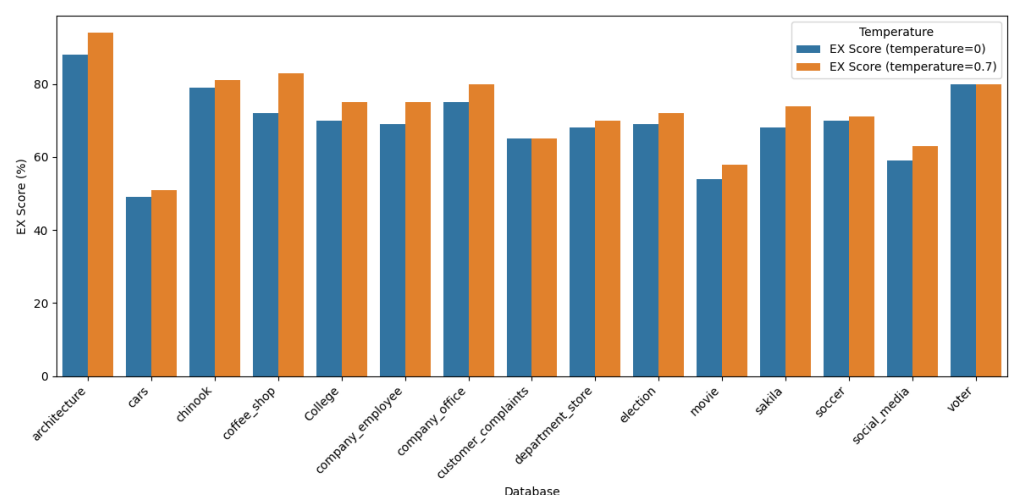


Figure 6. Success Rates by Database (GPT-4o + OpenAIEmbeddings(text-embedding-3-large) + ChromaDB).

As illustrated in Figure 6, the application of the “temperature scaling” strategy yielded a consistent improvement in Execution Accuracy (EX Score) across almost all databases tested. A comparative analysis reveals that the results obtained with a temperature of 0.7 (represented by orange bars) generally surpass those obtained with a deterministic setting of 0 (blue bars). This trend validates the hypothesis that introducing a degree of stochasticity enables the model to break out of deterministic error loops, allowing it to explore alternative and correct SQL syntax paths during the iterative repair phase.

Detailed observations from the experiments highlight specific areas of improvement. In databases containing complex schema structures, such as “architecture”, “coffee_shop”, and “college”, a marked increase in performance was observed. This suggests that for challenging scenarios where the model initially fails, the flexibility provided by a higher temperature is crucial for finding the correct solution in subsequent attempts. Furthermore, the strategy proved to be robust; in databases where baseline performance was already high, such as “voter” and “architecture”, the iterative repair mechanism either maintained the high scores or provided slight improvements, confirming that increased creativity does not introduce instability to the system.

4.4. Baseline Performance Comparison

In this initial phase of the study, the performance of three different large language models (GPT-4o, Gemini-1.5-Pro, Qwen-1.5) was compared on the standard RAG architecture without applying any data enrichment or hybrid query strategies. The purpose of these tests was to measure the models’ “raw” capabilities and their responses to different embedding/vector database combinations. The experimental results obtained are presented in Table 9.

Table 9. Comparison of LLM Performance.

LLM	Embedding + Vector Database	EX Score	Table Sim.	SQL-Sim.
gpt-4o	OpenAIEmbeddings(text-embedding-3-large) + ChromaDB	0.67	0.76	0.74
	OpenAIEmbeddings(text-embedding-3-large) + Pinecone	0.71	0.80	0.74
	OpenAIEmbeddings(text-embedding-3-large) + Qdrant	0.70	0.79	0.74
	CohereEmbeddings(embed-multilingual-v3.0) + ChromaDB	0.66	0.76	0.74
	CohereEmbeddings(embed-multilingual-v3.0) + Pinecone	0.70	0.80	0.74
	CohereEmbeddings(embed-multilingual-v3.0) + Qdrant	0.70	0.79	0.74
	HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1)+ChromaDB	0.53	0.63	0.70
	HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1) + Pinecone	0.67	0.77	0.73
	HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1) + Qdrant	0.68	0.78	0.74
gemini-1.5-pro-001	OpenAIEmbeddings(text-embedding-3-large) + ChromaDB	0.78	0.89	0.88
	OpenAIEmbeddings(text-embedding-3-large) + Pinecone	0.78	0.89	0.87
	OpenAIEmbeddings(text-embedding-3-large) + Qdrant	0.80	0.90	0.88
	CohereEmbeddings(embed-multilingual-v3.0) + ChromaDB	0.78	0.89	0.88
	CohereEmbeddings(embed-multilingual-v3.0) + Pinecone	0.79	0.89	0.88
	CohereEmbeddings(embed-multilingual-v3.0) + Qdrant	0.78	0.89	0.86
	HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1)+ChromaDB	0.76	0.88	0.88
	HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1)+Pinecone	0.79	0.89	0.87
	HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1)+Qdrant	0.79	0.89	0.88
Qwen-1.5-110B	OpenAIEmbeddings(text-embedding-3-large) + ChromaDB	0.65	0.81	0.73
	OpenAIEmbeddings(text-embedding-3-large) + Pinecone	0.64	0.79	0.73
	OpenAIEmbeddings(text-embedding-3-large) + Qdrant	0.66	0.80	0.73
	CohereEmbeddings(embed-multilingual-v3.0) + ChromaDB	0.63	0.79	0.73

CohereEmbeddings(embed-multilingual-v3.0) + Pinecone	0.65	0.80	0.72
CohereEmbeddings(embed-multilingual-v3.0) + Qdrant	0.65	0.80	0.73
HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1)+ChromaDB	0.59	0.75	0.71
HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1) + Pinecone	0.64	0.79	0.72
HuggingFaceEmbeddings(sentence-transformers-paraphrase-xlm-r-multilingual-v1) + Qdrant	0.63	0.79	0.72

The results obtained and model behaviours can be summarised as follows:

- **LLM Performance Analysis:** Looking at the scores, it was observed that the Gemini-1.5-Pro model performed better than its competitors in all metrics tested and achieved an EX score of 0.80. During the experimental process, it was found that the GPT-4o model was more affected by adverse situations where the number of schemas was insufficient or excessive. In contrast, the Gemini model managed the context better and was able to correct the SQLite query it could not find on the first attempt by the third attempt at most, thanks to its iterative repair mechanism. The open-source Qwen-1.5-110B model, despite having 110 billion parameters, lagged behind the GPT-4o and Gemini models in terms of success performance.
- **Impact of Embedding Models:** The results show that the best performance was achieved with the “text-embedding-3-large” model provided by OpenAI. This model has a vector length of 3072 dimensions and establishes strong semantic relationships between words thanks to its multilingual support. The “embed-multilingual-v3.0” model provided by Cohere has a vector length of 1072, while the open-source “sentence-transformers” model in the HuggingFace ecosystem produces 768-dimensional vectors. Table 9 shows that the sentence-transformers model scores lower than the others. This can be explained by the fact that a low vector size of 768 is insufficient to represent complex database schemas and cross-language relationships.
- **Vector Database Effect:** No significant difference in success scores was observed between the vector databases used (ChromaDB, Pinecone, Qdrant). In this context, it was determined that the main factor determining system performance is not the vector database, but rather the embedding model and Large Language Model (LLM) used.

4.5. Enhanced RAG (Optimisation) and GPT-4o Performance

In the second phase of the study, the effectiveness of the proposed “Data Enrichment” and “Advanced Embedding” strategies was measured. However, due to the discontinuation of the Gemini-1.5-Pro-001 and Qwen-1.5-110B models by service providers (model deprecation) and the termination of API access during the ongoing experiment period, the optimisation tests were concentrated on the GPT-4o model, which continued to be stable and accessible.

Accordingly, the detailed optimisation results obtained using the GPT-4o model are shown in Table 10. Upon examination of the table, the positive effect of the proposed enriched RAG method on GPT-4o is clearly evident:

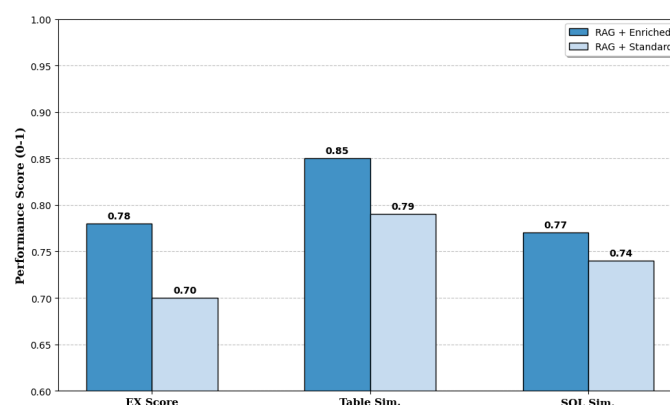
- **EX Score Increase:** The EX score, which was at 0.70 with the standard method, increased to 0.78 thanks to data enrichment in the OpenAIEmbeddings + Qdrant and OpenAIEmbeddings + Pinecone combinations.
- **Open Source Model Success:** Another noteworthy finding is that the open-source “ytuce-cosmos-turkish-e5-large” embedding model achieved an EX score of 0.78 in the enriched RAG structure, matching the success of commercial models (OpenAI).
- **Structural Accuracy:** The Table Similarity (Table Sim.) score reaching 0.85 proves that the model analyses column contents more accurately thanks to enriched data.

Table 10. Comparison of LLM Performance (Improved Method).

LLM	Embedding + Vector Database	EX Score	Table Sim.	SQL Sim.
gpt-4o	OpenAIEmbeddings(text-embedding-3-large) + ChromaDB	0.74	0.82	0.76
	OpenAIEmbeddings(text-embedding-3-large) + Pinecone	0.78	0.85	0.77
	OpenAIEmbeddings(text-embedding-3-large) + Qdrant	0.78	0.85	0.77
	VoyageAIEmbeddings(voyage-3-large) + ChromaDB	0.74	0.80	0.77
	VoyageAIEmbeddings(voyage-3-large) + Pinecone	0.77	0.84	0.77
	VoyageAIEmbeddings(voyage-3-large) + Qdrant	0.77	0.84	0.77
	HuggingFaceEmbeddings(ytu-ce-cosmos-turkish-e5-large) + ChromaDB	0.74	0.83	0.76
	HuggingFaceEmbeddings(ytu-ce-cosmos-turkish-e5-large) + Pinecone	0.78	0.85	0.76
	HuggingFaceEmbeddings(ytu-ce-cosmos-turkish-e5-large) + Qdrant	0.78	0.85	0.77

In summary, as seen in the comparison table presented in Table 10, the proposed method has increased the success of the GPT-4o model from 0.70 to 0.78. This result demonstrates that even if the model architecture remains unchanged, improving the quality of the context provided can bring the model's performance closer to that of the Gemini-1.5-Pro model (0.80), which was the leader at the beginning of the study.

The impact of these numerical improvements on key performance metrics is visualised in Figure 7. The graph presents a comparative analysis of the effects of the standard RAG method and the proposed enriched RAG method on the GPT-4o model, using the EX Score, Table Similarity, and SQL Similarity metrics.

**Figure 7.** Performance Comparison of Standard RAG and Enriched RAG Methods (GPT-4o).

4.6. Ablation Study: Impact of Individual Components

To respond to the need for quantifying the individual contributions of the proposed strategies, an ablation analysis was conducted. Instead of presenting a single final score, this section isolates the impact of each module, Data Enrichment, Hybrid Query, and Model Selection, on the Schema Retrieval performance using average F1 scores across all tested databases.

Impact of Data Enrichment: The first and most significant structural intervention was the inclusion of sample data rows in schema descriptions. As illustrated in Figure 8, this strategy provided a clear performance boost compared to the baseline (Standard RAG). The average F1 score increased from 0.740 to 0.798, proving that providing the LLM with actual data content reduces ambiguity significantly more than metadata alone.

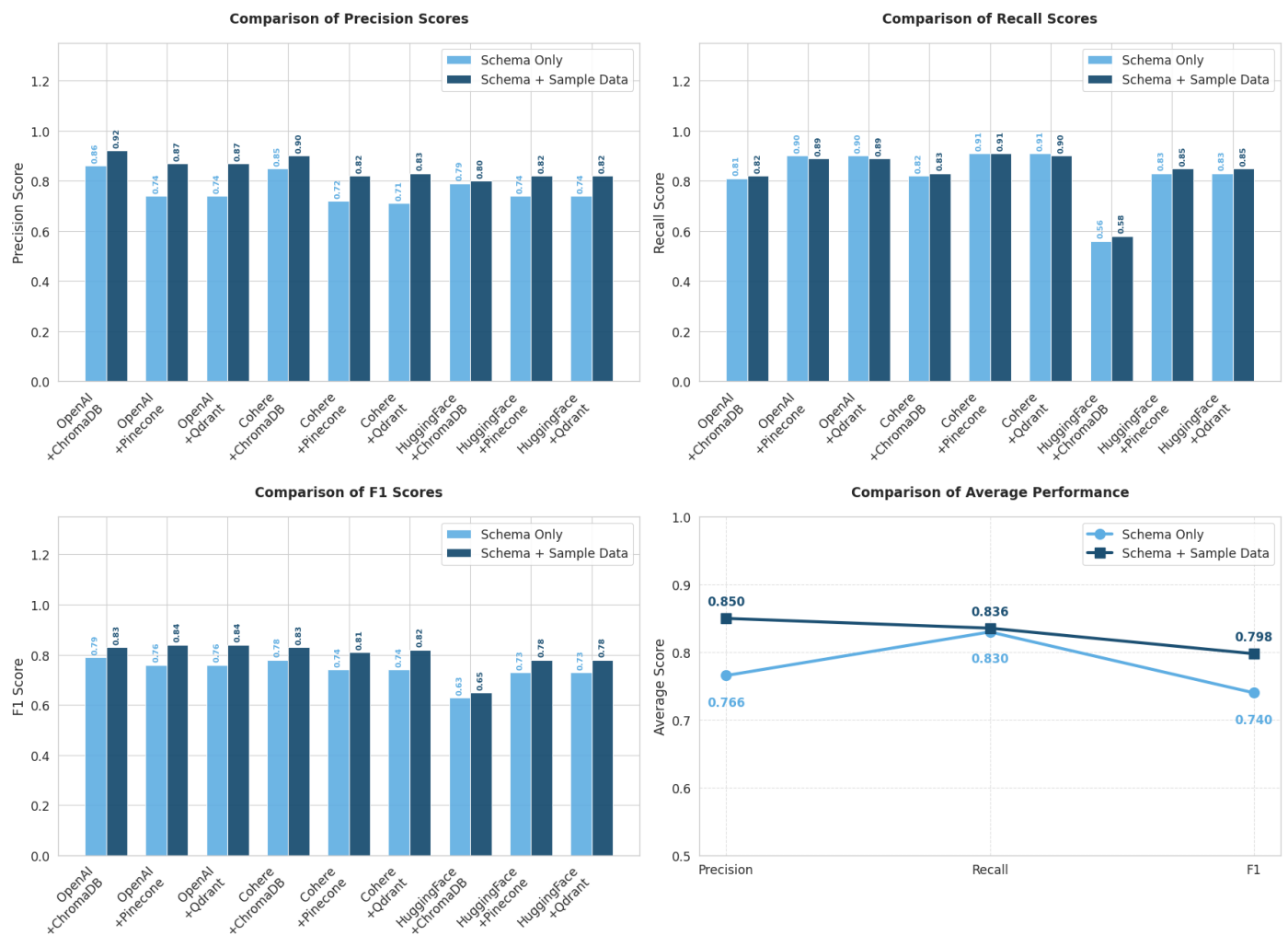
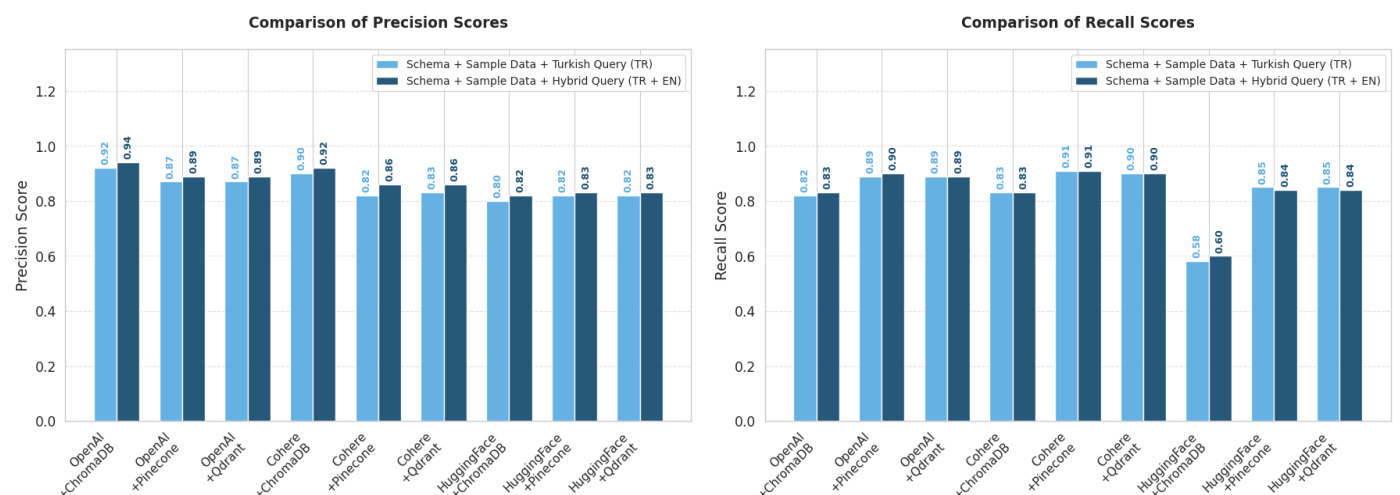


Figure 8. Impact of Data Enrichment on Precision, Recall, and F1 Scores.

Impact of Hybrid Query Strategy: The second layer of optimisation involved bridging the semantic gap between Turkish queries and English schemas. As shown in Figure 9, while the baseline Enriched RAG achieved an average F1 of 0.798, enabling the Hybrid (TR + EN) mode pushed the score to 0.810. Although the numerical margin appears small, the visualisation indicates a consistent gain across all metrics, acting as a semantic safety net for cross-lingual terminology.



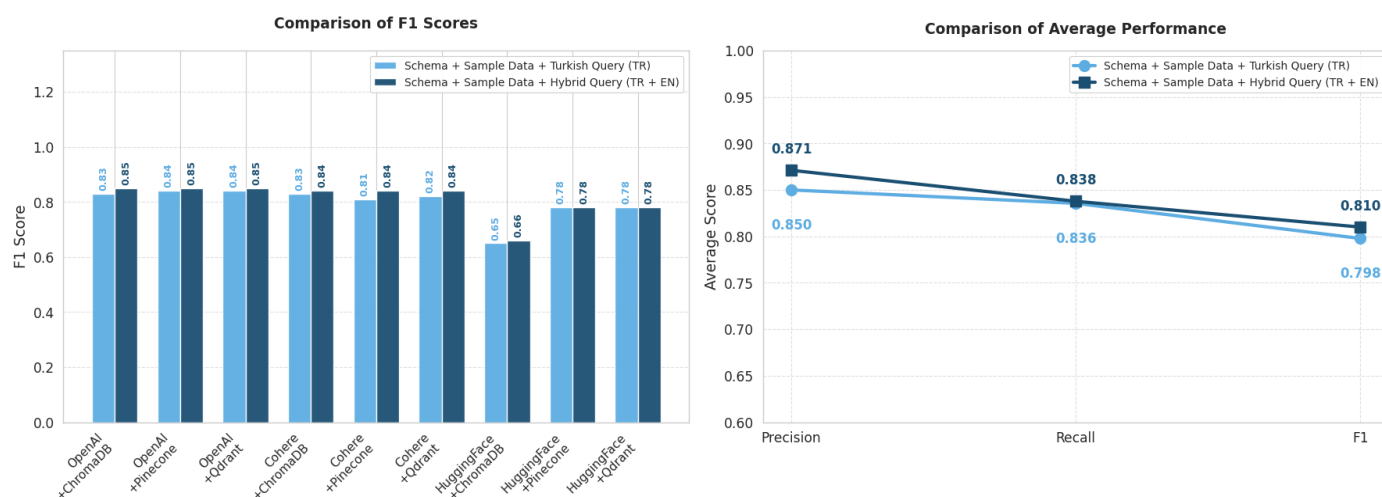


Figure 9. Impact of Hybrid Query Strategy on Performance Metrics.

Impact of Advanced Embedding Models: Switching to state-of-the-art embedding models resulted in the most dramatic architectural jump. As demonstrated in Figure 10, replacing standard models with the specialized “Voyage-3-large” model peaked the performance with an F1 score of 0.88. This confirms that while architectural improvements (Enrichment/Hybrid) are vital, the underlying quality of the embedding model remains a determining factor.

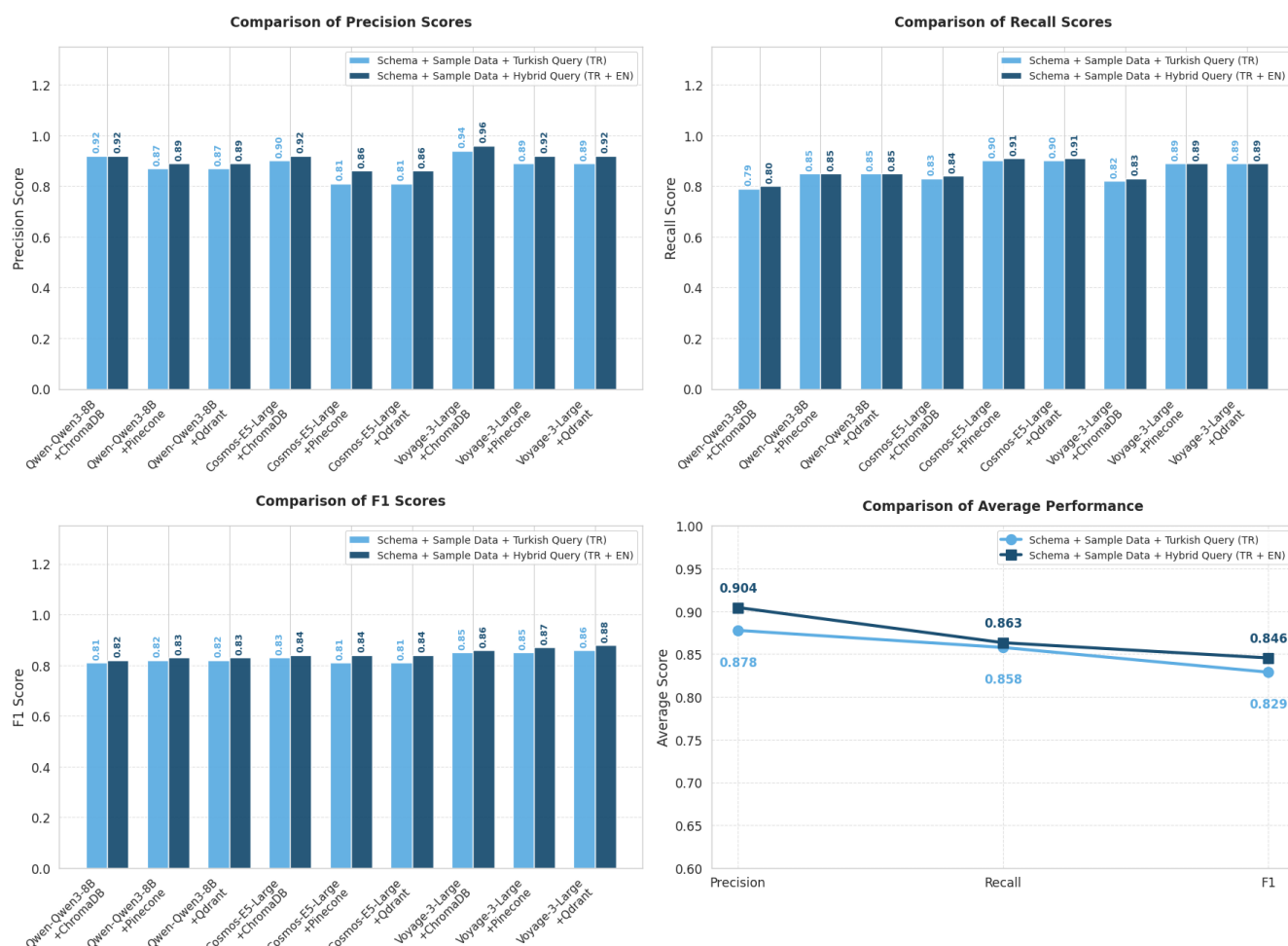


Figure 10. Comparison of Advanced Embedding Models.

4.7. The Effect of Schema Count and Complexity

The total of 1006 queries from 15 different databases used in the study are associated with schema counts ranging from 1 to 4. As the number of tables (schemas) associated with a query increases, the query's complexity increases, directly affecting the model's success.

Table 11 presents a performance analysis of the GPT-4o model (with standard RAG) broken down by schema count. The gaps in the table indicate that no queries with that schema count were found in the relevant database. For example, only queries with one and two schemas exist in the “coffee_shop” database. To obtain a general result, weighted averages were calculated, taking into account the query distribution in each database.

Table 11. Performance of Database Queries Based on the Number of Schemas (GPT-4o).

Databases	Total Query	1 Schema			2 Schema			3 Schema			4 Schema		
		Ct.	F1	EX	Ct.	F1	EX	Ct.	F1	EX	Ct.	F1	EX
coffee_shop	18	16	0.8	0.76	2	0.71	1						
election	68	38	0.79	0.79	30	0.86	0.57						
chinook	84	32	0.59	0.92	50	0.7	0.7	2	0.61	1			
architecture	17	8	0.85	1	7	0.69	0.9	2	0.74	0.33			
company_employee	16	11	0.96	0.79	1	0.7	1	4	0.71	0.44			
company_office	40	32	0.92	0.8	2	0.68	1	6	0.56	0.8			
voter	15	9	0.86	0.87	5	0.91	0.84	1	0.73	0			
customer_complaints	46	26	0.95	0.74	16	0.82	0.55	4	0.56	0.03			
movie	98	38	0.87	0.79	48	0.8	0.35	12	0.7	0.51			
social_media	76	22	0.93	0.7	48	0.78	0.55	6	0.68	0.5			
sakila	82	40	0.72	0.96	36	0.66	0.52	6	0.47	0.65			
soccer	106	68	0.77	0.75	34	0.76	0.53	4	0.6	0.19			
cars	82				54	0.76	0.59	21	0.74	0.34	7	0.71	0.06
department_store	88	46	0.73	0.65	28	0.72	0.61	12	0.66	0.55	2	0.68	0.83
college	170	104	0.62	0.75	44	0.61	0.67	20	0.58	0.55	2	0.57	0.44
WEIGHTED AVERAGE	1006	490	0.76	0.78	405	0.74	0.58	100	0.65	0.48	11	0.68	0.27

When examining the weighted average values in Table 11, a direct correlation was observed between schema access success (F1) and SQL generation success (EX). A dramatic decline in model performance was observed as the number of schemas associated with the queries increased:

- **1 Schematic Queries:** For simple queries associated with a single schema, the F1 score was 0.76, achieving an acceptable success rate with an EX score of 0.78.
- **Complex Queries (2–4 Schemas):** When the number of schemas increased to 2, the F1 score dropped to 0.74, while the EX score fell sharply to 0.58. In the most complex queries with 3 schemas, the F1 score dropped to 0.65; due to missing schema information, the model's ability to generate correct SQL decreased, causing the EX score to fall to 0.48.

This situation demonstrates that the standard RAG method struggles to retrieve the correct schemas (low F1) in scenarios requiring multi-table joins (JOIN), resulting in the failure of the GPT-4o model (low EX).

In the second phase of the study, Data Enrichment and Hybrid Query strategies were integrated into the system to prevent this performance loss and enhance the model's ability to understand context. The impact of these proposed improvements on performance across different schema counts is presented in Table 12.

Table 12 shows that the improvement in the F1 score is directly reflected in the EX score and that the proposed methods increase the system's success at every difficulty level:

- 1 Schematic Queries: The F1 score increased from 0.76 to 0.86; in parallel, the EX score increased from 0.78 to 0.88, representing a 11% increase.
- 2-Schema Queries: The significant increase in the F1 score (0.74 → 0.83) has raised the EX score from 0.58 to 0.66.
- 3 and 4 Schema Queries: In multi-table scenarios, where the system was most challenged, the F1 score for 3 schema queries rose from 0.65 to 0.79, thereby increasing the EX score from 0.48 to 0.62. Similarly, for 4-schema queries, the EX score increased from 0.27 to 0.46, enhancing the model's ability to handle complex contexts.

Table 12. Performance of Database Queries Based on the Number of Schemas (GPT-4o + Improved Method).

Databases	Total Query	1 Schema			2 Schema			3 Schema			4 Schema		
		Ct.	F1	EX	Ct.	F1	EX	Ct.	F1	EX	Ct.	F1	EX
coffee_shop	18	16	0.87	0.88	2	0.73	0.5						
election	68	38	0.88	0.82	30	0.95	0.67						
chinook	84	32	0.76	0.91	50	0.83	0.78	2	0.88	1			
architecture	17	8	1	1	7	0.71	1	2	0.9	0.5			
company_employee	16	11	0.88	1	1	0.67	1	4	0.73	0.5			
company_office	40	32	0.96	0.84	2	0.73	1	6	0.75	1			
voter	15	9	0.93	0.89	5	0.8	1	1	1	1			
customer_complaints	46	26	1	0.85	16	0.9	0.5	4	0.62	0.25			
movie	98	38	0.93	0.89	48	0.92	0.54	12	0.83	0.83			
social_media	76	22	0.95	0.68	48	0.79	0.48	6	0.8	0.33			
sakila	82	40	0.82	0.98	36	0.81	0.67	6	0.76	0.67			
soccer	106	68	0.88	0.9	34	0.81	0.53	4	0.65	0.25			
cars	82				54	0.79	0.69	21	0.84	0.57	7	0.71	0.43
department_store	88	46	0.83	0.76	28	0.79	0.79	12	0.78	0.83	2	0.83	1
college	170	104	0.77	0.93	44	0.82	0.75	20	0.77	0.5	2	0.4	0
WEIGHTED AVERAGE	1006	490	0.86	0.88	405	0.83	0.66	100	0.79	0.62	11	0.68	0.46

The performance difference between the standard RAG method and the proposed enriched RAG method is visualised in Figure 11.

As seen in the graph, the standard RAG method, represented by the dashed line, exhibits a rapid downward trend as the number of schemas increases; whereas the enriched method, represented by the solid line, demonstrates a more resilient and stable performance. The widening gap between the two graphs, particularly at points where the number of schemas increases, confirms that the proposed “sample data display” and “hybrid query” techniques play a critical role in both accessing the correct schema (F1) and generating the correct SQL (EX).

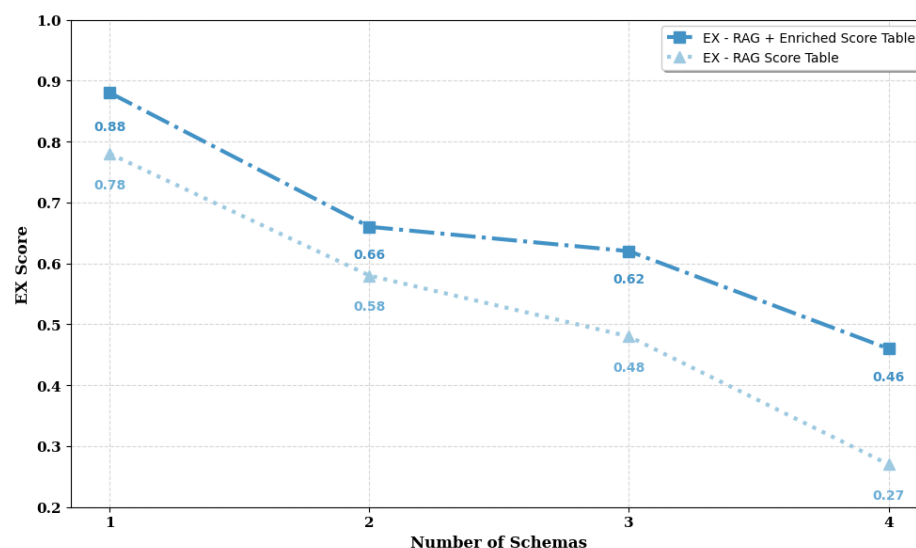


Figure 11. EX Score Comparison Based on Number of Schemas Using GPT-4o Model (Standard RAG vs. Enriched RAG).

In conclusion, the extensive experimental analysis conducted in this section reveals two key findings. First, while high-parameter commercial models like Gemini-1.5-Pro initially outperform others in baseline settings, the proposed data enrichment and hybrid query strategies can significantly boost the performance of other models (e.g., GPT-4o), enabling them to reach competitive accuracy levels without architectural changes. Second, the breakdown of performance by schema complexity demonstrates that the primary bottleneck in Text-to-SQL tasks is not merely SQL generation but the accurate retrieval of relevant schemas in multi-table scenarios. The proposed enriched RAG architecture effectively mitigates this bottleneck, providing a robust solution that maintains stability even as query complexity increases.

Building upon these empirical observations, we explicitly revisit and validate the research hypotheses formulated at the outset of this study:

Validation of H1: H1 posited that dynamic schema selection via hierarchical clustering would effectively minimize noise and yield high F1 scores in schema retrieval. The analysis in Section 4.1 validates this hypothesis. Specifically, the case study in Table 6 and the clustering distribution in Figure 5 demonstrate that the algorithm successfully identified a dynamic threshold to isolate relevant schemas (achieving 100% Recall for the sample query). Furthermore, the system achieved a high retrieval F1 score of 0.88 with advanced embedding models (refer to Table 8), confirming its efficacy in filtering out unrelated tables without relying on a fixed-k limit.

Validation of H2: H2 suggested that a hybrid (Turkish–English) query strategy would improve semantic matching in cross-lingual tasks. The ablation study presented in Section 4.6 supports this claim. As detailed in Figure 9, enabling the Hybrid Query mode increased the average F1 score from 0.798 to 0.810 across all metrics compared to the monolingual enriched baseline. This confirms that bridging the semantic gap between Turkish queries and English schemas acts as a critical safety net for retrieval accuracy.

Validation of H3: H3 claimed that enriching prompts with sample data and iterative self-correction would increase Execution Accuracy (EX) without fine-tuning. The experimental results strongly validate this hypothesis. First, Figure 8 shows that data enrichment alone raised the average F1 score from 0.740 to 0.798. Second, the impact of the iterative repair mechanism is evidenced in Figure 6, where temperature scaling significantly improved performance in complex databases

like “college” and “architecture”. Collectively, these optimizations raised the GPT-4o model’s EX score from 0.70 to 0.78, as summarized in Table 10, representing an 11% performance gain without model training.

5. Conclusions and Future Work

In this study, an innovative and high-performance RAG architecture has been developed to analyze natural language queries in Turkish, detect related database schemas, and automatically convert them into SQLite queries. Distinguishing itself from similar studies in the literature, this work presents a multi-layered optimization strategy that focuses not only on structural matching but also on data content and semantic context.

Comprehensive tests conducted on 15 different databases and 1006 queries during the experimental process have demonstrated the success of the proposed methods with concrete data. In the schema retrieval stage, while the F1 score remained at 0.79 using standard methods, it was increased to 0.88 with the integration of the “Data Enrichment” and “Hybrid Query” strategies proposed in this study, along with the transition to the “voyage-3-large” embedding model. This significant increase demonstrates that the language barrier and semantic gap have been effectively overcome.

In the SQL generation stage, baseline performance analyses revealed that the Gemini-1.5-Pro model achieved the most successful result with an EX score of 0.80, whereas the GPT-4o model remained at an EX score of 0.70. In the second phase of the study, the aim was to test the effectiveness of the proposed enriched RAG architecture; however, due to the deprecation of the Gemini-1.5-Pro-001 and Qwen-1.5-110B models by service providers during the experimental process, optimization efforts were concentrated on the GPT-4o model, which maintained accessibility and stability.

Optimization tests conducted on GPT-4o showed that the proposed methods increased the model’s performance from an EX score of 0.70 to 0.78. This finding proves that the proposed data enrichment and hybrid query techniques improved the model’s performance by approximately 11%, bringing it closer to the performance level of the initially leading model (Gemini) and establishing it in a competitive position. This result also demonstrates that LLM performance can be significantly enhanced by improving the provided context quality, even without altering the model architecture.

The implications of this study extend beyond simple query generation. The proposed system aligns with the broader trend of intelligent process automation, facilitating rapid data access for non-technical users and reducing the dependency on IT departments in decision-making processes [30–32]. By automating routine data retrieval tasks, this technology allows Database Administrators (DBAs) to shift their focus from writing ad hoc queries to high-level tasks such as database optimization, security, and architectural design. Furthermore, although this study focuses on SQLite, the prompt-based nature of the proposed RAG architecture supports a multi-dialect approach. The system can be easily generalized to generate queries for different SQL dialects (e.g., T-SQL, PL/SQL) or adapted to enterprise-level systems like PostgreSQL and Oracle, offering concrete prospects for industrial scalability.

However, this study has certain limitations that should be acknowledged. First, the “Error-Based Iterative Repair” mechanism, while significantly improving execution accuracy, introduces additional computational latency due to repeated API calls. This trade-off between accuracy and response time may pose challenges for real-time applications requiring millisecond-level feedback. Second, although open-source models (e.g., Qwen-1.5-110B) were evaluated in this study, the proposed architecture achieved its peak performance with commercial LLMs (GPT-4o). Consequently, the deployment of such high-performing systems entails operational costs and potential data privacy concerns com-

pared to fully offline open-source solutions. Third, this study focuses specifically on evaluating the impact of the proposed clustering and data enrichment strategies within a custom RAG architecture designed for the Turkish–English cross-lingual context, rather than benchmarking against general-purpose English-centric state-of-the-art (SOTA) prompting strategies such as DIN-SQL or DAIL-SQL. While these strategies represent significant advancements in prompt engineering, our primary objective was to isolate and measure the specific contribution of the proposed ‘Hierarchical Clustering’ and ‘Hybrid Query’ modules in overcoming the language barrier. Similarly, open-source frameworks like Vanna.ai or LlamaIndex were not directly compared to maintain full control over the retrieval pipeline and to rigorously test the proposed novel methodology without the abstraction layers of off-the-shelf tools. Future work will involve integrating SOTA prompting techniques into our cross-lingual architecture to observe potential synergistic effects. Finally, the experimental validation in this study was conducted on SQLite databases, consistent with the standard Spider and BIRD benchmarks used in the literature. While the core SQL logic generated by the proposed RAG architecture is largely transferable to other relational database management systems, specific dialect nuances (e.g., T-SQL, PL/SQL) and advanced enterprise features found in systems like Oracle or PostgreSQL remain to be verified in future adaptations.

In conclusion, this study serves as a significant resource for the literature, addressing the gap in datasets and methods within the fields of Turkish natural language processing and Text-to-SQL. Future work aims to develop fine-tuned, indigenous embedding models using datasets specifically prepared for the Text-to-SQL task, rather than relying on general-purpose embedding models. Additionally, plans include testing the proposed hybrid architecture on larger-scale and more complex database management systems such as PostgreSQL and MySQL.

Author Contributions: Conceptualization, M.B. (Mehmet Bozdemir) and M.B. (Metin Bilgin); methodology, M.B. (Mehmet Bozdemir); software, M.B. (Mehmet Bozdemir); validation, M.B. (Mehmet Bozdemir) and M.B. (Metin Bilgin); formal analysis, M.B. (Mehmet Bozdemir); investigation, M.B. (Mehmet Bozdemir); resources, M.B. (Mehmet Bozdemir) and M.B. (Metin Bilgin); data curation, M.B. (Mehmet Bozdemir); writing—original draft preparation, M.B. (Mehmet Bozdemir); writing—review and editing, M.B. (Mehmet Bozdemir) and M.B. (Metin Bilgin); visualization, M.B. (Mehmet Bozdemir); supervision, M.B. (Metin Bilgin); project administration, M.B. (Metin Bilgin); funding acquisition, M.B. (Metin Bilgin). All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Bursa Uludağ University Science and Technology Centre (BAP) under Grant FGA-2025-2123.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available on request from the corresponding author. The study utilized open-source datasets (BIRD and Spider), and the Turkish translated versions of the queries along with the generated “Related_schemas” data are not publicly available due to ongoing research but can be provided upon reasonable request.

Acknowledgments: The authors acknowledge the support provided by the Scientific Research Projects Coordination Unit of Bursa Uludag University.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CBOW	Continuous Bag-of-Words
CRUD	Create, Read, Update, Delete
EX	Execution Accuracy
F1	F1 Score (Harmonic Mean of Precision and Recall)
FN	False Negative
FP	False Positive
GPT	Generative Pre-trained Transformer
LLM	Large Language Model
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
RDBMS	Relational Database Management System
SQL	Structured Query Language
TN	True Negative
TP	True Positive

References

1. Minaee, S.; Mikolov, T.; Nikzad, N.; Chenaghlu, M.; Socher, R.; Amatriain, X.; Gao, J. Large Language Models: A Survey. *arXiv* **2024**, arXiv:2402.06196. Available online: <https://arxiv.org/abs/2402.06196> (accessed on 4 January 2026).
2. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention Is All You Need. *arXiv* **2017**, arXiv:1706.03762. Available online: <https://arxiv.org/abs/1706.03762> (accessed on 4 January 2026).
3. Naveed, H.; Khan, A.U.; Qiu, S.; Saqib, M.; Anwar, S.; Usman, M.; Akhtar, N.; Barnes, N.; Mian, A. A Comprehensive Overview of Large Language Models. *arXiv* **2023**, arXiv:2307.06435. Available online: <https://arxiv.org/abs/2307.06435> (accessed on 4 January 2026).
4. Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, M.; Wang, H. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv* **2023**, arXiv:2312.10997. Available online: <https://arxiv.org/abs/2312.10997> (accessed on 4 January 2026).
5. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-T.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv* **2020**, arXiv:2005.11401. Available online: <https://arxiv.org/abs/2005.11401> (accessed on 4 January 2026).
6. Wang, M.; Shafran, I.; Soltau, H.; Han, W.; Cao, Y.; Yu, D.; El Shafey, L. Retrieval Augmented End-to-End Spoken Dialog Models. In Proceedings of the 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Seoul, Republic of Korea, 14–19 April 2024; Institute of Electrical and Electronics Engineers (IEEE): New York, NY, USA, 2024; pp. 12056–12060. <https://doi.org/10.1109/icassp48485.2024.10447448>.
7. Ma, L.; Zhang, R.; Han, Y.; Yu, S.; Wang, Z.; Ning, Z.; Zhang, J.; Xu, P.; Li, P.; Ju, W.; et al. A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge. *arXiv* **2023**, arXiv:2310.11703. Available online: <https://arxiv.org/abs/2310.11703> (accessed on 4 January 2026).
8. Hong, Z.; Yuan, Z.; Zhang, Q.; Chen, H.; Dong, J.; Huang, F.; Huang, X. Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL. *arXiv* **2024**, arXiv:2406.08426. Available online: <https://arxiv.org/abs/2406.08426> (accessed on 4 January 2026).
9. Zhu, X.; Li, Q.; Cui, L.; Liu, Y. Large Language Model Enhanced Text-to-SQL Generation: A Survey. *arXiv* **2024**, arXiv:2410.06011. Available online: <https://arxiv.org/abs/2410.06011> (accessed on 4 January 2026).
10. Shi, L.; Tang, Z.; Zhang, N.; Zhang, X.; Yang, Z. A Survey on Employing Large Language Models for Text-to-SQL Tasks. *ACM Comput. Surv.* **2025**, *58*, 1–37. <https://doi.org/10.1145/3737873>.
11. Kanburoglu, A.B.; Tek, F.B. TUR2SQL: A Cross-Domain Turkish Dataset for Text-to-SQL. In *UBMK 2023—Proceedings: 8th International Conference on Computer Science and Engineering, Burdur, Türkiye, 13–15 September 2023*; Institute of Electrical and Electronics Engineers Inc.: New York, NY, USA, 2023; pp. 206–211. <https://doi.org/10.1109/UBMK59864.2023.10286686>.

12. Kanburoğlu, A.B.; Tek, F.B. Text-to-SQL: A methodical review of challenges and models. *Turk. J. Electr. Eng. Comput. Sci.* **2024**, *32*, 403–419. <https://doi.org/10.55730/1300-0632.4077>.
13. Wang, C.; Tatwawadi, K.; Brockschmidt, M.; Huang, P.S.; Mao, Y.; Polozov, O.; Singh, R. Robust Text-to-SQL Generation with Execution-Guided Decoding. *arXiv* **2018**, arXiv:1807.03100. Available online: <https://arxiv.org/abs/1807.03100> (accessed on 4 January 2026).
14. Kumar, R.; Amar, A.; Dibbu, R.; Harsola, S.H.; Subrahmaniam, V.; Modi, A. BookSQL: A Large Scale Text-to-SQL Dataset for Accounting Domain. In *Human Language Technologies (Volume 1: Long Papers), Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics, Mexico City, Mexico, 16–21 June 2024*; Association for Computational Linguistics: Mexico City, Mexico, 2024; pp. 497–516.
15. Rushdy, M.R.A.; Thayasivam, U. Application of Noise Filter Mechanism for T5-Based Text-To-SQL Generation. In *MERCon, Proceedings of the Moratuwa Engineering Research Conference, Moratuwa, Sri Lanka, 9–11 November 2023*; Institute of Electrical and Electronics Engineers Inc.: New York, NY, USA, 2023; pp. 95–100. <https://doi.org/10.1109/MERCon60487.2023.10355492>.
16. Demirkiran, F.; Coşkun, A.K.; Kömeçoğlu, Y.; Kömeçoğlu, B.B.; Güven, R. Enhancing Text-to-SQL Conversion in Turkish: An Analysis of LLMs with Schema Context. In *Proceedings of the 2024 9th International Conference on Computer Science and Engineering (UBMK), Antalya, Türkiye, 26–28 October 2024*; IEEE: New York, NY, USA, 2024; pp. 1–5. <https://doi.org/10.1109/UBMK63289.2024.10773403>.
17. Usta, A.; Karakayali, A.; Ulusoy, Ö. DBTagger: Multi-task learning for keyword mapping in NLIDBs using bi-directional recurrent neural networks. *Proc. Vldb Endow.* **2021**, *14*, 813–821. <https://doi.org/10.14778/3446095.3446103>.
18. Zhang, K.; Lin, X.; Wang, Y.; Zhang, X.; Sun, F.; Cen, J.; Tan, H.; Jiang, X.; Shen, H. ReFSQL: A Retrieval-Augmentation Framework for Text-to-SQL Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*; Association for Computational Linguistics: Singapore, 2023; pp. 664–673.
19. Wang, B.; Ren, C.; Yang, J.; Liang, X.; Bai, J.; Chai, L.; Yan, Z.; Zhang, Q.-W.; Yin, D.; Sun, X.; et al. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025), Abu Dhabi, United Arab Emirates, 19–24 January 2025*; pp. 540–557. Available online: <https://github.com/wbbeyourself/MAC-SQL> (accessed on 4 January 2026).
20. Pourreza, M.; Rafiei, D. DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models. *arXiv* **2024**, arXiv:2402.01117. Available online: <https://arxiv.org/abs/2402.01117> (accessed on 4 January 2026).
21. Vu, T.; Balasubramaniam, T.; Nayak, R.; Keretna, S. Improved Productivity with AI Models for SQL Tasks: A Case Study. In *Computer Science & Information Technology (CS & IT)*; Academy and Industry Research Collaboration Center (AIRCC): Chennai, India, 2024; pp. 21–33. <https://doi.org/10.5121/csit.2024.142402>.
22. Li, J.; Hui, B.; Qu, G.; Yang, J.; Li, B.; Li, B.; Wang, B.; Qin, B.; Geng, R.; Huo, N.; et al. Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs. *arXiv* **2023**, arXiv:2305.03111. Available online: <https://arxiv.org/abs/2305.03111> (accessed on 4 January 2026).
23. Gao, D.; Wang, H.; Li, Y.; Sun, X.; Qian, Y.; Ding, B.; Zhou, J. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *arXiv* **2023**, arXiv:2308.15363. Available online: <https://arxiv.org/abs/2308.15363> (accessed on 4 January 2026).
24. Spider_Bird: Combined Spider and BIRD Dataset for Text-to-SQL. Available online: https://huggingface.co/datasets/star-rysky9959/spider_bird (accessed on 1 October 2024).
25. Almeida, F.; Xexéo, G. Word Embeddings: A Survey. *arXiv* **2019**, arXiv:1901.09069. Available online: <https://arxiv.org/abs/1901.09069> (accessed on 4 January 2026).
26. Mikolov, T.; Sutskever, I.; Chen, K.; Corrado, G.; Dean, J. Distributed Representations of Words and Phrases and their Compositionality. *arXiv* **2013**, arXiv:1310.4546. Available online: <https://arxiv.org/abs/1310.4546> (accessed on 4 January 2026).
27. Mikolov, T.; Chen, K.; Corrado, G.; Dean, J. Efficient Estimation of Word Representations in Vector Space. *arXiv* **2013**, arXiv:1301.3781. Available online: <https://arxiv.org/abs/1301.3781> (accessed on 4 January 2026).
28. Yang, H.; Zhang, M.; Wei, D. SRAG: Speech Retrieval Augmented Generation for Spoken Language Understanding. In *ISRITI 2023, Proceedings of the 6th International Seminar on Research of Information Technology and Intelligent Systems, Batam, Indonesia, 11 December 2023*; Institute of Electrical and Electronics Engineers Inc.: New York, NY, USA, 2023; pp. 279–283. <https://doi.org/10.1109/ISRITI60336.2023.10467285>.
29. Lenssen, L.; Schubert, E.; Krivošija, A.; Schubert, E.; Lang, A.; Hess, S. 5 Cluster Analysis. In *Fundamentals*; De Gruyter: Berlin, Germany, 2022; pp. 179–248. <https://doi.org/10.1515/9783110785944-005>.
30. Dospinescu, O.; Buraga, S. Integrated ERP Systems—Determinant Factors for Their Adoption in Romanian Organizations. *Systems* **2025**, *13*, 667. <https://doi.org/10.3390/systems13080667>.

31. Lankut, E.; Warner-Söderholm, G.; Alon, I.; Minelgaité, I. Big Data in Leadership Studies: Automated Machine Learning Model to Predict Preferred Leader Behavior Across Cultures. *Businesses* **2024**, *4*, 696–722. <https://doi.org/10.3390/businesses4040039>.
32. Abasova, A.; Mamedova, K.; Alekperova, L. Selection of software tools for automation of business planning processes. *Econ. Transp. Complex* **2023**, *41*, 78. <https://doi.org/10.30977/ETK.2225-2304.2023.41.78>.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.